INNOPOLIS UNIVERSITY

SNE MASTER TRACK

2026

COURSE: [S26] DevOps and Security

DevOps and Security Lab 2 REPORT

On

Software configuration management tools (base lab)

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 03.02.2026

Index

Abstract…….. **3**

Task 1 - Prerequisites…… ……**4-11**

Task 2 - SCM Theory…….. **11-15**

Task 3 - Play with SCM……..… **15-56**

Bonus 1 - Ansible collections…….. **56-65**

Bonus 2 - Ansible AWX…….…. **66-80**

References……..…. **81-82**

Abstract

This lab project designs, implements, and validates a complete end-to-end DevOps automation workflow in a real-world environment. The work begins with **Task 1 (Prerequisites)**, where a production and a development/control environment are created using two separate Virtual Machines. The production environment is ubuntu-prod and the development and control node is fedora-dev. Although the fedora-dev VM uses Ubuntu Server, this is consistent with the task instructions, as the operating systems are mentioned as examples (e.g.). This step establishes a secure and automation-friendly foundation by creating SSH-based remote access, key-based authentication, and an Ansible-ready environment.

Task 2 (SCM Theory) provides a theoretical analysis of Ansible as a Software Configuration Management (SCM) tool. The role of Ansible's directory structure, roles, playbooks, variables, handlers, templates, and configuration files is explained in detail. Special emphasis is given to variable precedence, defaults vs. vars vs. *group_vars*, and how to keep environment-based configuration separate, which is an important part of production-grade automation design.

Task 3 (Play with SCM - Ansible) puts theory into practice. A structured Ansible project is created following best-practices in the Control node, where inventory, roles, playbooks, *group_vars*, and Ansible Vault are arranged separately. Three scenarios – Users, Firewall (UFW), and NTP (Chrony) - are implemented. Out of these, Users and Firewall were the light scenario and NTP was the advanced scenario. The automation uses SSH key-based access, Vault-based secret management, idempotent execution, handler-driven service restart, Jinja2 template, loop, conditional logic, block/rescue/always, and *run_once/delegate_to*. The stability and reliability of the automation are proven through real execution, check-mode, syntax-check, and full run.

Bonus 1 (Ansible Galaxy Collections) focuses on external dependency management. The *community.general* collection from Ansible Galaxy is selected, installed, and integrated into the project. The *community.general.ufw* module is used in firewall automation to prove that the Galaxy collection is not just installed, but actually consumed. The project is kept reproducible and clean using *requirements.yml* and *collections_paths*, so that the same environment can be easily rebuilt in the future.

Bonus 2 (AWX Deployment) upgrades Ansible automation to a graphical and enterprise-style interface. Ansible AWX is successfully deployed on a K3s-based single-node Kubernetes cluster using the AWX Operator. Kubernetes namespace, CRD, secret-based admin credential, *NodePort* service exposure, and pod lifecycle are verified during deployment. Operational issues (disk pressure, pod eviction) caused by resource constraints are identified and recovery is demonstrated through cleanup and rollout restart. Finally, a robust Proof of Concept (PoC) was presented with *HTTP 200 OK* response at all levels of *NodePort, ClusterIP, Pod IP* and *port-forward*. Overall, this lab was not limited to just showing the use of Ansible or AWX, but rather it showcased a real DevOps mindset where secure access, modular automation, environment-based configuration, reproducibility, troubleshooting and verification were implemented together.

Task 1 - Prerequisites

Prepare at least two different guest systems (VMs), e.g. an Ubuntu Server (aka "prod env") and a Fedora (aka "dev env"). Then choose a Software configuration management (SCM) tool to automate system preparations on those guests instances. Ansible is the default choice and the most popular tool for SCM. We recommend you to use Ansible. By the way, there are some alternatives that you are able to choose:

- **SaltStack**
- **Puppet**
- **Chef**
- **Another one...**

Solution:

Virtual Machine Setup:

Two separate Virtual Machines (VMs) have been prepared for this lab, one for the Production environment and the other for the Development and Control environment. The Production environment uses a VM named ubuntu-prod, which has Ubuntu Server installed and is essentially acting as a target/production node. On the other hand, the VM named fedora-dev is used as the Development environment and is also used for Ansible Control Node, Kubernetes (*K3s*) host, and AWX deployment. Although the Development VM is named fedora-dev, it actually has the Ubuntu Server operating system installed on it. The word “e.g.” (example) is used in the task description, which indicates that the given OSes are not mandatory but are mentioned as examples.

VM-1: ubuntu-prod (Production Environment):

Updating the package list:

```
hasib@ubuntu-prod:~$ sudo apt update
[sudo] password for hasib:
Hit:1 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
150 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Before installing new packages in Ubuntu, the system's package list is updated to include the latest repository information and bring the system up-to-date.

Installing OpenSSH Server:

```
hasib@ubuntu-prod:~$ sudo apt update
sudo apt install -y openssh-server
[sudo] password for hasib:
Hit:1 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
150 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ncurses-term openssh-client openssh-sftp-server ssh-import-id
Suggested packages:
  keychain libpam-ssh monkeysphere ssh-askpass molly-guard
The following NEW packages will be installed:
  ncurses-term openssh-server openssh-sftp-server ssh-import-id
The following packages will be upgraded:
  openssh-client
1 upgraded, 4 newly installed, 0 to remove and 149 not upgraded.
```

OpenSSH Server is installed because it allows remote login from the host machine via network. *-y* is used so that it does not prompt for confirmation during installation. Ansible does not work without SSH, because Ansible works on remote nodes using SSH.

Starting SSH service and enabling auto-start:

```
hasib@ubuntu-prod:~$ sudo systemctl enable --now ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/syste
md/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink /etc/systemd/system/sshd.service → /usr/lib/systemd/system/ssh.
service.
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service → /usr/
lib/systemd/system/ssh.service.
```

SSH is automatically started when the system boots with enable and SSH service is started with *--now*. SSH service is linked with Multi-user target through *systemd*, as a result, SSH is not stopped even after reboot.

Checking if SSH service is running:

```
hasib@ubuntu-prod:~$ sudo systemctl status ssh --no-pager
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enab
led)
     Active: active (running) since Wed 2026-01-28 03:26:02 MSK; 6s ago
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 7387 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS
)
   Main PID: 7389 (sshd)
      Tasks: 1 (limit: 4545)
     Memory: 1.2M (peak: 1.5M)
        CPU: 48ms
     CGroup: /system.slice/ssh.service
             └─7389 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jan 28 03:26:02 ubuntu-prod systemd[1]: Starting ssh.service - OpenBSD Sec…r...
Jan 28 03:26:02 ubuntu-prod sshd[7389]: Server listening on 0.0.0.0 port 22.
Jan 28 03:26:02 ubuntu-prod sshd[7389]: Server listening on :: port 22.
Jan 28 03:26:02 ubuntu-prod systemd[1]: Started ssh.service - OpenBSD Secu…ver.
Hint: Some lines were ellipsized, use -l to show in full.
```

The SSH service was running successfully *active* and ready to accept connections on *Port 22*.

Finding the IP Address of the VM:

```
hasib@ubuntu-prod:~$ ip -4 a | grep inet
    inet 127.0.0.1/8 scope host lo
    inet 192.168.145.129/24 brd 192.168.145.255 scope global dynamic noprefixro
ute ens33
```

The IP has been extracted, for remote login via SSH and to add the host to the Ansible inventory file.

Updating the package list:

```
hasib@fedora-dev:~$ sudo apt update
[sudo] password for hasib:
Hit:1 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
150 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Installing OpenSSH Server:

```
hasib@fedora-dev:~$ sudo apt install -y openssh-server
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ncurses-term openssh-client openssh-sftp-server ssh-import-id
Suggested packages:
  keychain libpam-ssh monkeysphere ssh-askpass molly-guard
The following NEW packages will be installed:
  ncurses-term openssh-server openssh-sftp-server ssh-import-id
The following packages will be upgraded:
  openssh-client
1 upgraded, 4 newly installed, 0 to remove and 149 not upgraded.
Need to get 1,738 kB of archives.
After this operation, 6,743 kB of additional disk space will be used.
Get:1 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 openssh-clien
t amd64 1:9.6p1-3ubuntu13.14 [906 kB]
Get:2 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 openssh-sftp-
server amd64 1:9.6p1-3ubuntu13.14 [37.3 kB]
Get:3 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 openssh-serve
r amd64 1:9.6p1-3ubuntu13.14 [510 kB]
Get:4 http://ru.archive.ubuntu.com/ubuntu noble/main amd64 ncurses-term all 6.4+
20240113-1ubuntu2 [275 kB]
Get:5 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 ssh-import-id
```

OpenSSH Server has been installed so that remote login to *fedora-dev* VM from host machine is possible.

After installing the server, enable it:

```
hasib@fedora-dev:~$ sudo systemctl enable --now ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/system
d/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink /etc/systemd/system/sshd.service → /usr/lib/systemd/system/ssh.s
ervice.
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service → /usr/l
ib/systemd/system/ssh.service.
```

Because of using this command, even though the system reboots with enable, SSH starts automatically, and the SSH service is started at that moment with *--now.* As a result, the SSH service is associated with the system boot.

Checking if the SSH service is running:

```
hasib@fedora-dev: $ sudo systemctl status ssh --no-pager
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-01-28 02:24:21 MSK; 11s ago
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 6224 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 6226 (sshd)
      Tasks: 1 (limit: 4545)
     Memory: 1.2M (peak: 1.6M)
        CPU: 52ms
     CGroup: /system.slice/ssh.service
             └─6226 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jan 28 02:24:21 fedora-dev systemd[1]: Starting ssh.service - OpenBSD Secur…r...
Jan 28 02:24:21 fedora-dev sshd[6226]: Server listening on 0.0.0.0 port 22.
Jan 28 02:24:21 fedora-dev sshd[6226]: Server listening on :: port 22.
Jan 28 02:24:21 fedora-dev systemd[1]: Started ssh.service - OpenBSD Secure…ver.
Hint: Some lines were ellipsized, use -l to show in full.
```

This shows that the SSH service is active and running properly and is ready to accept connections on *Port 22.*

Finding the IP Address of the fedora-dev VM:

```
       valid_lft forever preferred_lft forever
hasib@fedora-dev:~$ ip -4 a | grep inet
    inet 127.0.0.1/8 scope host lo
    inet 192.168.145.130/24 brd 192.168.145.255 scope global dynamic noprefixroute ens33
```

The IP address has been extracted for SSH connection and added as host in Ansible inventory file.

SCM Tool Selection:

Fedora-dev VM is used as Control Node VM and Ansible is run from this VM to manage other VMs like (ubuntu-prod).

Ansible Setup (Control Node):

Fedora-dev VM is used as Ansible Control Node in this lab.

Installing Ansible on fedora-dev VM:

VM-2: fedora-dev:

```
hasib@fedora-dev:~$ sudo apt install -y ansible
[sudo] password for hasib:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ansible is already the newest version (9.2.0+dfsg-0ubuntu5).
ansible set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 149 not upgraded.
```

This command is used to install Ansible. The *-y* flag is used to not ask for confirmation during the installation.

Verifying that Ansible is installed:

```
hasib@fedora-dev:~$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/hasib/.ansible/plugins/modules', '/usr/share/ansibl
e/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/hasib/.ansible/collections:/usr/share/ansible/collectio
ns
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Jan  8 2026, 11:30:50) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
```

The output confirms that Ansible is properly installed and ready to use.

- Ansible Core: 2.16.3
- Python Version: 3.12.3

SSH Connectivity and Key-Based Login:

Testing SSH:

```
hasib@fedora-dev:~$ ssh hasib@192.168.145.129
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-37-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

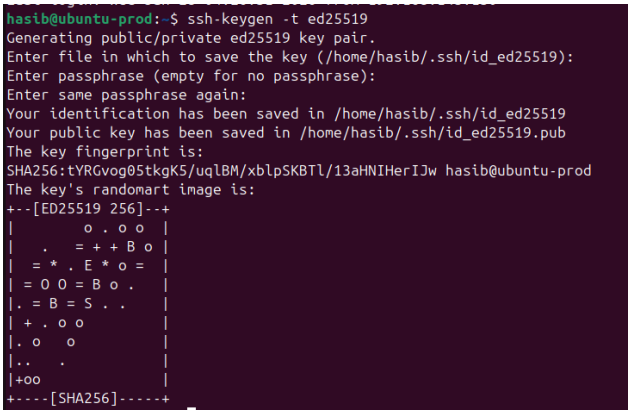
148 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Wed Jan 28 04:26:31 2026 from 192.168.145.130
```

The output shows Ubuntu login banner and shell access. This indicates that the network connectivity is fine and the SSH service is running properly. As a result, the login from fedora-dev to ubuntu-prod has been successful.

Creating SSH Key:

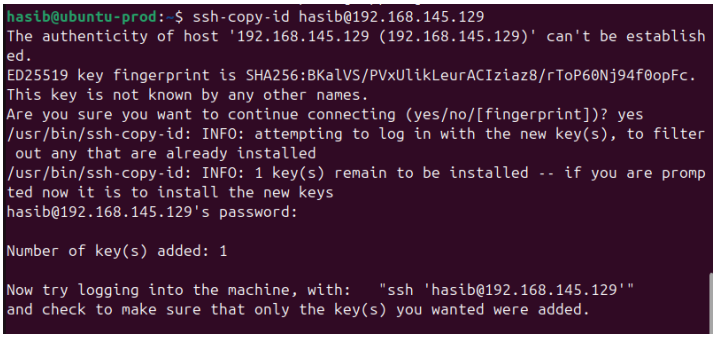


Key-based authentication is used to enable password-less SSH during Ansible automation.

Creating SSH Keys on ubuntu-prod VM:



Copying SSH Key - so Ansible can SSH without password:



This command copied the public key to the ubuntu-prod VM and installed the key using the SSH password once.

Completing Ansible-ready SSH authentication:

```
hasib@fedora-dev:~$ ssh-copy-id hasib@192.168.145.130
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'hasib@192.168.145.130'"
and check to make sure that only the key(s) you wanted were added.
```

This command copied the public key to the ubuntu-prod VM and installed the key using the SSH password once.

Creating Ansible Inventory:

Creating Inventory Folder:

```
hasib@ubuntu-prod:~$ mkdir -p ~/lab2/inventory
```

Inventory File:

```
hasib@fedora-dev:~$ nano ~/lab2/inventory/hosts.ini
```

The *~/lab2/inventory/hosts.ini* or inventory file has to be kept in a separate and structured manner using the nano text editor.



In this configuration, the Ansible inventory file is organized so that production and development environments are clearly separated. The [prod] group represents the production environment, where ubuntu-prod is a logical host name that Ansible uses internally. This host is set to the IP address of the actual VM, with *ansible_host=192.168.145.129*, and *ansible_user=hasib* is specified to make it clear which user is used during the SSH connection.

Ansible Ping is run where:

```
hasib@fedora-dev:~$ ansible -i ~/lab2/inventory/hosts.ini all -m ping
ubuntu-dev | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
ubuntu-prod | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

The development VM is placed in the *[dev] group* based on the user used during SSH. This makes it easy for Ansible to separate prod and dev environments and run group-wise automation.

Conclusion for Task 1:

Task 1 uses two virtual machines to separate production (*ubuntu-prod*) and development (*fedora-dev*) environments for Ansible automation, with *fedora-dev* configured as the Ansible Control Node. Two Ubuntu Server virtual machines were provisioned to represent production and development environments. Although different Linux distributions (e.g., Ubuntu and Fedora) can be used, Ubuntu was chosen for both environments to maintain consistency and simplify configuration management. Ansible was used as the Software Configuration Management (SCM) tool to automate system preparation across both guest systems. SSH servers were installed and enabled on both VMs, allowing remote management over the network, which was a fundamental requirement for Ansible to work. Ansible was properly installed and verified on the Control Node, and password-less, non-interactive automation was ensured by setting up SSH key-based authentication. An inventory file was created and placed in separate groups for production and development hosts, allowing environment-wise automation. Finally, the Ansible ping test was successfully run to prove that all target VMs are reachable and automation-ready from the Ansible Control Node.

Task 2 - SCM Theory

Briefly answer for the following questions what is and for what:

1. **ansible** directory:

ansible.cfg

inventory folder

roles folder

tasks

defaults/group_vars

handlers

templates

files

vars

meta folder

playbooks folder

Solution:

1. ansible directory:

Solution:

ansible.cfg

This is the main configuration file for Ansible. It defines how Ansible will work, which inventory to use, SSH user, *sudo* management, output format, etc. Using it eliminates the need to repeatedly enter command-line options.

inventory folder

This folder contains a list of managed hosts or servers. Here, servers can be divided into different groups (e.g. *prod, dev, web, db*). This makes it easy to apply the same playbook to multiple servers.

roles folder

Roles are the most important structure in Ansible. Each role represents a specific task (e.g. create user, install nginx). Using roles makes the automation code reusable and clean.

tasks

This folder contains the main tasks of the role. Such as package install, service start/stop, file copy, etc. Ansible executes the tasks written here sequentially.

defaults / group_vars

defaults:

The default variable value of the role is stored here. These have the lowest priority and can be easily overridden.

group_vars:

Variables are stored for a specific host group. Such as different configurations for prod and dev environments.

handlers

Handlers are tasks that run only when a change occurs. Usually used to restart or reload the service. This avoids unnecessary work.

templates

The Templates folder contains Jinja2 templates. These create dynamic configuration files using variables. Different configs can be created for different environments with the same template.

files

Static files are stored here. Such as scripts, certificates, static config files. These files are copied directly to the remote host.

vars

Role-specific or hard-coded variables are stored here. These have higher priority and are usually not overridden.

meta folder

This folder contains role metadata. Such as role dependency, author or version information. If a role depends on another role, it is mentioned here.

playbooks folder

Playbook is the main entry point of Ansible automation. It is determined which role will run on which host. It orchestrates the entire automation process.

2. Research, list and explain the most important parameters from ansible.cfg in your opinion.

Solution:

Inventory: Specifies which inventory file or folder to use.

remote_user: Specifies which user Ansible will use to connect to the remote system.

become: Enables or disables privilege escalation (*sudo*).

become_method: Specifies which method to use, such as *sudo, su*, etc.

host_key_checking: If set to False, the SSH confirmation prompt is not displayed on the new host.

roles_path: Specifies where Ansible will look for roles.

stdout_callback: Controls how Ansible will display output (yaml, json, etc.).

3. Research and explain the difference between *roles/.../vars* , *roles/.../defaults* and *playbook/.../group_vars* variables definitions.

Solution:

Ansible provides the facility to define variables in different places. The three most commonly used places are *roles/.../defaults, roles/.../vars* and *group_vars* of playbook or inventory. Their function, importance and purpose of use are different.

The variables in the *roles/.../defaults* directory are the default values assigned to a role. The variables placed here are given the lowest priority. Its main purpose is to keep the role reusable and flexible. If a role is used elsewhere and the user does not explicitly define any variable, then the value given in defaults will be used. For example, if *http_port: 80* is given in defaults in a role, then the web service will run on *port 80* unless the user writes anything separately. However, if the user wants, he can easily override this value from another place. Therefore, defaults basically works as a safe fallback configuration and according to best practice, the general settings of the role should be kept here.

The variables in the *roles/.../vars* directory are much more powerful or high-priority. These are usually variables that are essential to the core logic of the role and that can break the role if

changed. These variables are usually not overridden and are intentionally set strictly by the role author. For example, if a role uses a specific internal path or system user, it can be placed in vars. Variables defined in vars take precedence over defaults and are usually used very carefully in production-grade roles, because additional vars make the role rigid.

The *group_vars* of a playbook or inventory are mainly used to separate the configuration according to the environment or host group. Here, the variables are not tied to a specific role, but rather to a specific host group (such as *prod, dev, webservers, dbservers).* The biggest advantage of this is that the same role or playbook can work differently in different environments, without any code changes. For example, the database password or production-specific settings can be stored in *group_vars/prod.yml*, and the test credentials can be used in *group_vars/dev.yml*. This makes *group_vars* infrastructure clean, scalable, and environment-aware.

4. Learn and explain Ansible variable precedence.

Solution:

Ansible variable precedence means that if a variable is defined in more than one place with the same name, Ansible will take it first. Sometimes, playbooks do not work as expected due to incorrect precedence.

Variables in Ansible can come from many sources, such as *defaults*, *inventory*, grou*p_vars*, *playbook vars, role vars, extra-vars*, etc. There is a specific precedence order among these sources. The general rule is that the more "*explicit*" or user-defined the variable is, the higher its priority.

At the lowest level are role defaults. These act as fallback values and defaults are always overridden if the same variable is defined elsewhere. Just above these are inventory variables and *group_vars*, which determine the configuration according to a specific host or group. These are more powerful than defaults, because they are environment-specific.

Next come vars or *vars_files* in the playbook. Variables defined directly in the playbook usually override inventory or *group_vars*, because they are explicitly specified during playbook execution. At the next level are role vars (*roles/.../vars*), which have a much higher priority than defaults and are generally difficult to override.

At the top level are extra variables (*extra-vars*), which are passed from the command line using the -e option. These have the highest precedence in Ansible. That is, if the same variable is present in *defaults*, *group_vars*, *playbook vars*, and *extra-vars*, the value in extra-vars will ultimately be used.

Task 3 - Play with SCM

1. Play with SCM tool to automate system preparations on those. You are able to create and provide your own ideas or follow to some suggestions:

Light:

Ansible Vault (definitely a good choice)

IPtables (close all machine ports exclude ssh, http, https)

Docker containers deployment

set the default shell

Linux users and groups management (to create, to change permissions...)

install some applications on the target hosts (apt, rpm, pip...)

copy & backup files

restart services

pulling git repository

...

A kinda advanced:

NTP

Apache Web Server

Kubernetes helm chart deployment

Nginx & PHP-FPM & MariaDB & WordPress installation and configuration

ssh keys management.

certificates creation (let's encrypt)

resize volume partitions on the target hosts

make a network configuration between several virtual hosts (vlans, IPs, routing, firewall ...)

deploy a postgresdb or other DBMS

deploy rabbitmq, redis or kafka

deploy Jenkins

...

Try to follow the next Ansible best practices:

use *templates/jinja2*

separate your configuration files into **inventory/roles/playbooks** files

do not expose sensitive date and secrets in plain text!

play with *handlers*

try to avoid a raw shell command in Ansible tasks as much as you can!

follow to **Idempotence** principle!

play with Ansible variables priorities (precedence)

do not hard code variables values in tasks

use loops

use bloks

if possible, try to play with different Ansbile features like import_tasks, when, become,

become_user, loop_control, delegate_to, run_once, tags, ignore_errors, retries, delay, with_items, notify, set_fact and others

DO NOT COPY/PAST EXISTING ANSIBLE PLAYBOOKS! If you use any code as a base and

example, just mention it and list what you've changed and how you understand it

To pass the lab, complete two "light" and one "advanced" SCM usage scenarios.

Bonus: use Ansible Molecula and Ansible Lint to test your role before to run the corresponding playbook.

If you use an other SCM tool, replace the task goals according to your tool

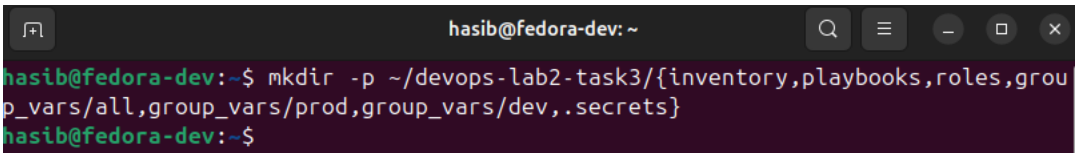
specifications.

2. Provide the link to git repository with all your labs configurations.

Solution:

Setup & Project Files in Control Node:

Creating Project Structure:

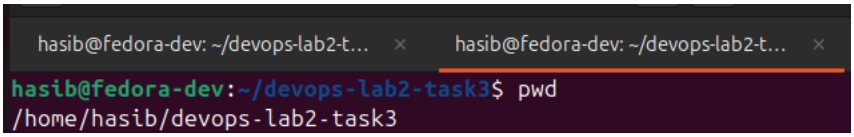


This directory structure is built following Ansible best practices, where each part has a clearly defined role. The inventory/ directory holds host and group definitions, playbooks/ holds the main automation logic, and roles/ holds reusable and modular configuration. Similarly, group-wise variables are separated using *group_vars/,* which simplifies variable precedence and environment-based configuration. The .secrets/ directory holds the Vault password to keep sensitive information safe.

Project Structure / Working Directory:

Check the working directory on the control node (fedora-dev):



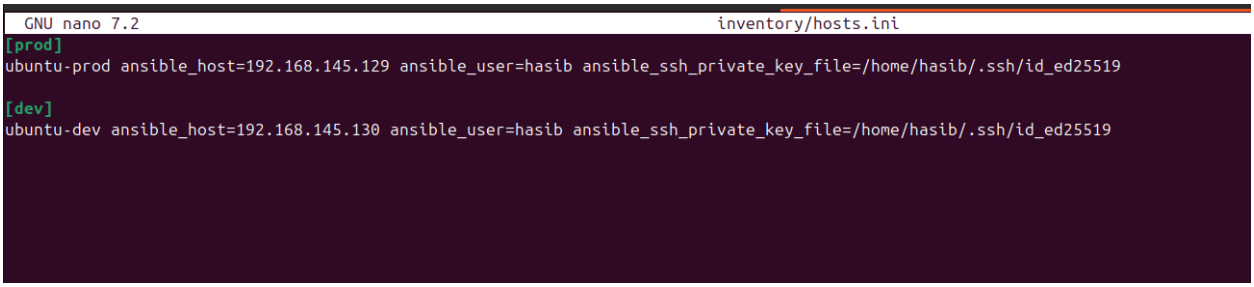


This directory is used as the root directory for the entire Ansible project. It contains *inventory/roles/playbooks*.

Creating the Inventory file:

Inventory Setup - hosts.ini:





Here, environment-based grouping is done using *[prod]* and *[dev],* which separates production and development hosts. For each host, the real IP address is specified with *ansible_host*, the user to be used when *SSHing* is specified with *ansible_user,* and the addition of

ansible_ssh_private_key_file enables SSH key-based authentication. As a result, there is no need to enter a separate password when *SSHing*.

Inventory verify:

```
hasib@fedora-dev:~/devops-lab2-task3$ cat inventory/hosts.ini
[prod]
ubuntu-prod ansible_host=192.168.145.129 ansible_user=hasib ansible_ssh_private_key_file=/home/hasib/.ssh/id_ed25519

[dev]
ubuntu-dev ansible_host=192.168.145.130 ansible_user=hasib ansible_ssh_private_key_file=/home/hasib/.ssh/id_ed25519
```

Inventory is written so that Ansible can easily distinguish hosts by environment and automation is completely non-interactive. Production and development servers are separated with *[prod]* and *[dev],* the real IP of each host is specified with *ansible_host*, and SSH key-based login is set using *ansible_user* and *ansible_ssh_private_key_file*. This allows group-wise automation to be reliably run without a password.

Creating *ansible.cfg*:

```
hasib@fedora-dev:~/devops-lab2-task3$ nano ansible.cfg
```

```
 GNU nano 7.2                                                ansible.cfg
[defaults]
inventory = inventory/hosts.ini
stdout_callback = yaml
retry_files_enabled = False
host_key_checking = True
interpreter_python = auto_silent
roles_path = ./roles
vault_password_file = .secrets/vault_pass.txt

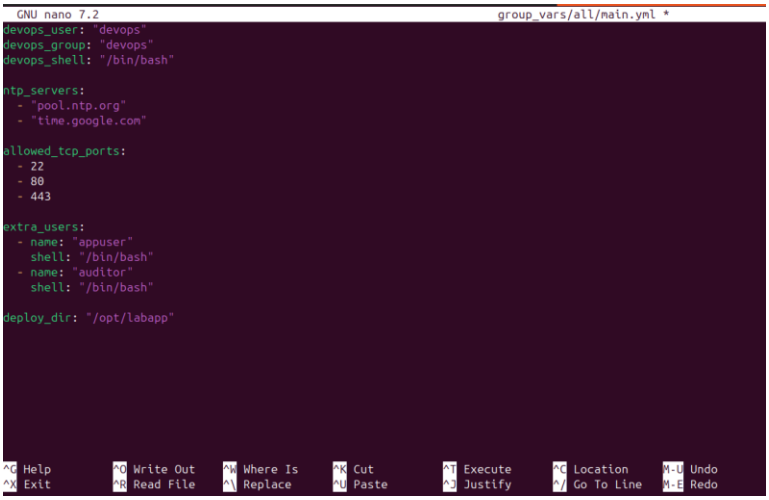
[privilege_escalation]
become = True
become_method = sudo
become_ask_pass = False
```

ansible.cfg is used because this file provides Ansible with project-level defaults for the entire project. This eliminates the need to repeatedly specify *-i inventory/hosts.ini*, the output is readable in YAML format, *sudo* (become) automation works normally, and the Vault password is automatically loaded.

group_vars:

The following configuration has been added to the *group_vars/all/main.yml* file using the nano editor.

```
hasib@fedora-dev:~/devops-lab2-task3$ nano group_vars/all/main.yml
```



Here the *group_vars/all/main.yml* file is used to define common (global) variables for all hosts. The default user configuration is set with *devops_user, devops_group* and *devops_shell*, which is applicable to all environments. The *ntp_servers* section is intentionally kept in such a way that it can be overridden later from *group_vars/prod* to show variable precedence. Similarly, *allowed_tcp_ports* is used to determine which TCP ports will be open in the firewall, and the *extra_users* list is used to make it easy to create multiple users through a loop. Finally, the *deploy_dir* variable is kept as a usable path for privilege escalation or deployment related tasks.

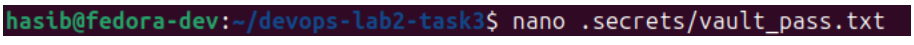
The following configuration has been added to the *group_vars/prod/main.yml* file using the nano editor:





Here the *group_vars/prod/main.yml* file is used to provide a separate configuration for production hosts only. In this file, *ntp_servers* is redefined, so that it overrides the same variable in *group_vars/all/main.yml*. This avoids hard-coding environment-specific settings and clearly shows Ansible's variable precedence where the value in the prod group takes precedence over the common value for all hosts.)

Creating a vault password file:



```
hasib@fedora-dev:~/devops-lab2-task3$ chmod 600 .secrets/vault_pass.txt
hasib@fedora-dev:~/devops-lab2-task3$
```

The Vault password file is created separately in the *fedora-dev* VM in *.secrets/vault_pass.txt* so that the key to unlock the Vault is never hardcoded in the code or repository. By giving *chmod 600*, this file can only be read and used by the owner, no one else has access. This reduces the risk of accidental leaks and keeps sensitive information safe.

Creating Vault encrypted vars:

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-vault view group_vars/all/vault.yml \
  --vault-password-file .secrets/vault_pass.txt
```

Vault encrypted variables are used to securely manage sensitive data in the fedora-dev VM. The *group_vars/all/vault.yml* file is encrypted with ansible-vault and accessed using *.secrets/vault_pass.txt*, so that passwords or sudo-related secrets are never in plaintext.

Light - 1

Create users role files:

```
hasib@fedora-dev:~/devops-lab2-task3$ mkdir -p roles/users/{tasks,handlers,defaults,vars}
hasib@fedora-dev:~/devops-lab2-task3$
```

The tasks are divided into separate task files using *import_tasks in* the *roles/users/tasks/main.yml* file, such as user creation and demo features.

After adding the following config to the *roles/users/tasks/main.yml* file using the nano editor:

```
hasib@fedora-dev:~/devops-lab2-task3$ nano roles/users/tasks/main.yml
```

```
  GNU nano 7.2                                    roles/users/tasks/main.yml
---
- name: Import user creation tasks
  ansible.builtin.import_tasks: create_users.yml

- name: Import demo feature tasks
  ansible.builtin.import_tasks: demo_features.yml
```

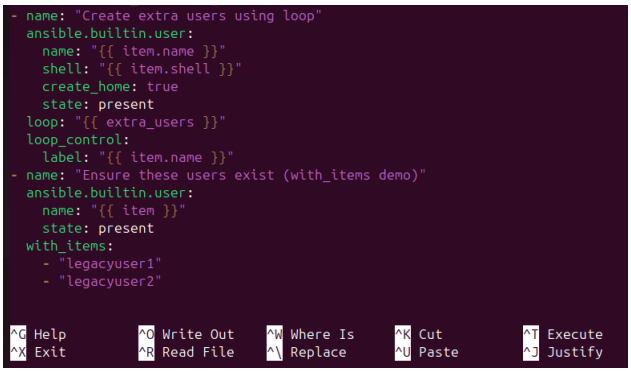
This role is designed to showcase the core features of Ansible in a clean and practical way. It demonstrates how to create users and groups, use encrypted passwords stored in Vault, handle multiple users with *loop/with_items*, and demo advanced features like *become_user, delegate_to, run_once*. The role is created on a fedora-dev VM and follows the standard role structure (tasks, handlers, defaults, vars).

After adding the following config to the *roles/users/tasks/create_users.yml* file using the nano editor:

```
hasib@fedora-dev:~/devops-lab2-task3$ nano roles/users/tasks/create_users.yml
```

The following *create_users.yml* file briefly demonstrates some of Ansible's important features. It uses *set_fact* to create a *password_hash* from a secret stored in the vault, eliminating the need for a plaintext password. All user and group creation is completely variable-driven, with no hardcode. OS-specific work is demonstrated with when conditions, and loop, *loop_control*, and legacy *with_items* syntax are used to create multiple users.

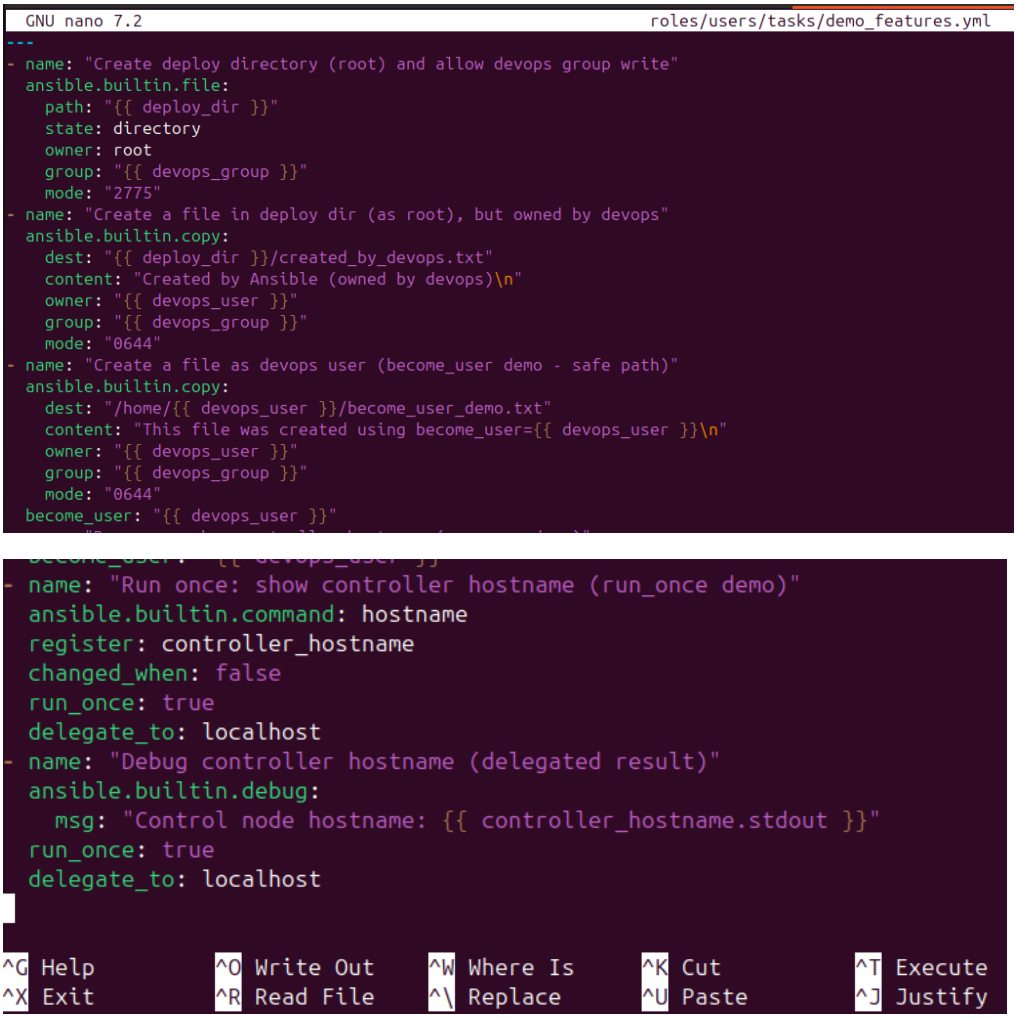




After adding the following config to the nano *roles/users/tasks/demo_features.yml* file using the nano editor:

```
hasib@fedora-dev:~/devops-lab2-task3$ nano roles/users/tasks/demo_features.yml
```

This *demo_features.yml* file is intended to briefly demonstrate some real-world DevOps patterns in Ansible.

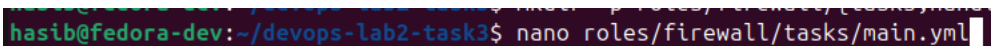


Here, the file creation is shown by changing the privilege context using *become_user,* the task is executed on the control node using *delegate_to*, and the efficiency of not having to repeat the same task in a multi-host setup using *run_once* is shown. Directory permission and ownership are also set.

Light - 2

Firewall role files (UFW module):

After adding the following config to the *roles/firewall/tasks/main.yml* file using the nano editor:



This firewall role is created in the *fedora-dev* VM to make firewall configuration fully automated and production-safe using UFW.

```
 GNU nano 7.2                                          roles/firewall/tasks/main.yml
---
- name: "Install ufw"
  ansible.builtin.apt:
    name: ufw
    state: present
    update_cache: true
- name: "Allow required TCP ports (loop) - MUST run before enabling deny"
  community.general.ufw:
    rule: allow
    port: "{{ item }}"
    proto: tcp
  loop: "{{ allowed_tcp_ports }}"
  loop_control:
    label: "Allow TCP {{ item }}"
  notify: reload ufw
- name: "Enable UFW (idempotent)"
  community.general.ufw:
    state: enabled
  notify: reload ufw
- name: "Set default incoming policy to deny (after allow rules)"
  community.general.ufw:
    direction: incoming
    policy: deny
  notify: reload ufw
```

Here we show how to use notify and handler to reload UFW only when the configuration changes, which ensures idempotency.

After creating the *roles/firewall/handlers/main.yml* file using the nano editor:

```
hasib@fedora-dev:~/devops-lab2-task3$ nano roles/firewall/handlers/main.yml
```

Here, *ufw* has been installed first, then the required TCP ports have been allowed through a loop using the *allowed_tcp_ports* variable. Firewall has been enabled and the default incoming policy deny has been set, so that the server is secure.

```
 GNU nano 7.2                                        roles/firewall/handlers/main.yml
---
- name: Reload UFW
  ansible.builtin.command: ufw reload
  register: ufw_reload
  changed_when: false
  failed_when: false
```

Advanced

NTP role files:

```
hasib@fedora-dev:~/devops-lab2-task3$ mkdir -p roles/ntp/{tasks,handlers,templates,defaults}
hasib@fedora-dev:~/devops-lab2-task3$
```

After setting the following configuration in the *roles/ntp/templates/chrony.conf.j2* file using the nano editor:

```
hasib@fedora-dev:~/devops-lab2-task3$ nano roles/ntp/templates/chrony.conf.j2
```

The NTP (Chrony) role is designed to meet advanced requirements, because instead of writing a static config, the *pool ... iburst* lines are automatically generated from the *ntp_servers* list using a Jinja2 loop in the *chrony.conf.j2* template. As a result, the entire Chrony config is updated by changing variables in one place.

```
 GNU nano 7.2                                  roles/ntp/templates/chrony.conf.j2 *


{% for s in ntp_servers %}
pool {{ s }} iburst
{% endfor %}

driftfile /var/lib/chrony/chrony.drift
makestep 1.0 3
rtcsync
logdir /var/log/chrony
```

Also, the *group_vars* precedence demo works, even though all groups have default NTP servers, for production hosts, the value override from *group_vars/prod* applies to a different NTP server on prod.

This *handlers/main.yml* file is kept to properly manage the lifecycle of the Chrony service.

```
hasib@fedora-dev:~/devops-lab2-task3$ nano roles/ntp/handlers/main.yml
hasib@fedora-dev:~/devops-lab2-task3$
```

Whenever there is a change in the NTP configuration or template, this handler is triggered via notify and the chrony service is restarted.

```
 GNU nano 7.2
---
- name: Restart chrony
  ansible.builtin.service
    name: chrony
    state: restarted
```

This *tasks/main.yml* file is designed to keep the NTP role clean and modular.

```
hasib@fedora-dev:~/devops-lab2-task3$ nano roles/ntp/tasks/main.yml
hasib@fedora-dev:~/devops-lab2-task3$
```

Here, Chrony's tasks have been divided into three separate task files - install, configure, and *verify* - and added using *import_tasks*.

```
  GNU nano 7.2                                                  roles/ntp/tasks/main.yml
---
- name: Import chrony install tasks
  ansible.builtin.import_tasks: install.yml

- name: Import chrony configure tasks
  ansible.builtin.import_tasks: configure.yml

- name: Import chrony verify tasks
  ansible.builtin.import_tasks: verify.yml
```

This *install.yml* task is designed to install Chrony and is made OS-aware.

```
hasib@fedora-dev:~/devops-lab2-task3$ nano roles/ntp/tasks/install.yml
```

Here in the file below, chrony package is installed on Ubuntu system using apt module, and idempotency is maintained by giving *state: present. update_cache: true* ensures package list update. Finally, *when: ansible_facts['os_family'] == "Debian"* condition is used, so that this task runs only on Debian family system.

```
  GNU nano 7.2                                                  roles/ntp/tasks/install.yml
---
- name: "Install chrony"
  ansible.builtin.apt:
    name: chrony
    state: present
    update_cache: true
  when: ansible_facts['os_family'] == "Debian"
```

This *configure.yml* file handles Chrony's configuration and service management.

```
hasib@fedora-dev:~/devops-lab2-task3$ nano roles/ntp/tasks/configure.yml
hasib@fedora-dev:~/devops-lab2-task3$
```

The following template module deploys the config file from *chrony.conf.j2*, where the *ntp_servers* from *group_vars* are applied. When the config changes, the Restart chrony handler is triggered via notify. Then the Chrony service is enabled and started using the service task, so that the service is automatically started after boot.

```
  GNU nano 7.2                                    roles/ntp/tasks/configure.yml
---
- name: "Deploy chrony config (template)"
  ansible.builtin.template:
    src: chrony.conf.j2
    dest: /etc/chrony/chrony.conf
    owner: root
    group: root
    mode: "0644"
  notify: restart chrony

- name: "Enable/start chrony"
  ansible.builtin.service:
    name: chrony
    enabled: true
    state: started
```

This *verify.yml* file verifies that *Chrony/NTP* is working properly.

```
hasib@fedora-dev:~/devops-lab2-task3$ nano roles/ntp/tasks/verify.yml
hasib@fedora-dev:~/devops-lab2-task3$
```

Here, several verification tasks are put together using blocks. First, the chronyc sources *-v* command is run with retry and delay, so that the service is ready when it is found and the command is retried until it succeeds. Then, the NTP sources found with debug are displayed. This is used to confirm that the NTP setup is indeed active and functional, as a production-style verification.

```
---
- name: "Verify NTP sources (block/rescue demo)"
  block:
    - name: "Wait for chrony sources to be available"
      ansible.builtin.command: chronyc sources -v
      register: chrony_sources
      retries: 5
      delay: 3
      until: chrony_sources.rc == 0
      changed_when: false

    - name: "Show chrony sources"
      ansible.builtin.debug:
        var: chrony_sources.stdout_lines
```

```
  rescue:
    - name: "Chrony verification failed - show service status (ignore_errors demo)"
      ansible.builtin.command: systemctl status chrony --no-pager
      register: chrony_status
      changed_when: false
      failed_when: false
    - name: "Debug chrony status"
      ansible.builtin.debug:
        var: chrony_status.stdout_lines
      failed_when: false
      changed_when: false
  always:
    - name: "Always: print which servers are configured"
      ansible.builtin.debug:
        msg: "Configured NTP servers: {{ ntp_servers }}"
```

Control Node:

community.general (UFW module):

This step is done because in the Firewall role we use the *community.general.ufw* module, which is not included by default in the Ansible core and is part of the *community.general* collection. The collection is installed with the following command, so that UFW rules can be managed in an idempotent and clean way and it is possible to avoid raw iptables or shell-command based configuration.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-galaxy collection install community.general
Starting galaxy collection install process
Nothing to do. All requested collections are already installed. If you want to reinstall them, consider using `--force`.
```

collection was previously installed, so there was no need to reinstall it.

Main Playbook in Control Node:

Creating *playbook/site.yml*:

The *playbooks/site.yml* file is designed as an entry point for the entire project, where all roles are orchestrated from one place.

```
hasib@fedora-dev:~/devops-lab2-task3$ nano playbooks/site.yml
hasib@fedora-dev:~/devops-lab2-task3$
```

Here, by giving *hosts: all*, the automation is applied to both production and development hosts in the inventory. become: true is used because user creation, firewall configuration, and NTP setup are not possible without root permission.

In the *vars_files* section, non-secret global variables (*main.yml*) and encrypted sensitive variables (*vault.yml*) are loaded separately, so that both security and clarity are maintained. Then, three roles are added to the roles section - users and firewall as a light scenario, and *ntp* as an advanced scenario. Tags are given to each role, so that a specific role or scenario can be run separately if necessary.

```
 GNU nano 7.2                                                playbooks/site.yml
---
- name: "Lab2 Task3 - Ansible automation scenarios"
  hosts: all
  become: true
  vars_files:
    - "{{ playbook_dir }}/../group_vars/all/main.yml"
    - "{{ playbook_dir }}/../group_vars/all/vault.yml"
  roles:
    - { role: users, tags: ["light", "users"] }
    - { role: firewall, tags: ["light", "firewall"] }
    - { role: ntp, tags: ["advanced", "ntp"] }
```

Light Scenario

Vault Secrets (Ansible Vault):

Vault file view:

In this step, the secrets stored in Vault are verified using ansible-vault view. The main purpose of using Vault is to ensure that no secrets are ever in plaintext form in the project or Git repository.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-vault view group_vars/all/vault.yml --vault-password-file .secrets/vault_pass.txt
```

Here *devops_user_password* is used when creating the devops Linux user, where the password is set by hashing it within the role.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-vault view group_vars/all/vault.yml \
  --vault-password-file .secrets/vault_pass.txt
devops_user_password: "ChangeThisToStrongPassword@123"
ansible_become_password: "109898"
ansible_password: "109898"
```

ansible_become_password is required when using sudo or become, when root privileges are required. *Ansible_password* was originally intended for password-based SSH login. However, it is no longer needed as key-based authentication has been adopted.

Initial Issue: *sshpass* missing (password-based SSH):

This error occurred which was later resolved with key-based inventory.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -m ping all --vault-password-file .secrets/vault_pass.txt
```

This issue occurred when running ansible -m ping with password-based SSH. If Ansible is trying to connect using an SSH password, the sshpass tool is often required. The fedora-dev system did not have *sshpass* installed, which is why this error occurred.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -m ping all --vault-password-file .secrets/vault_pass.txt
ubuntu-prod | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
ubuntu-dev | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
hasib@fedora-dev:~/devops-lab2-task3$
```

Later, we switched from password-based SSH to key-based authentication. Setting the private key path in Inventory no longer required *sshpass*. As a result, SSH became more secure, automation became completely non-interactive, and production-grade Ansible workflows worked properly.

Ping with inventory:

In this step, the entire setup has been verified by running ansible *-m ping* using inventory and Vault.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m ping --vault-password-file .secrets/vault_pass.txt
```

Getting *SUCCESS: pong* from both *ubuntu-prod* and *ubuntu-dev* hosts means that the inventory worked properly, SSH connectivity is fine, and Ansible was able to communicate with the target hosts without any problems.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m ping --vault-password-file .secrets/vault_pass.txt
ubuntu-prod | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
ubuntu-dev | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
hasib@fedora-dev:~/devops-lab2-task3$
```

test (whoami):

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "whoami" --become --vault-password-file .secrets/vault_pass.txt
```

In this test, the *whoami* command was run with *--become*, and the output for both hosts was root. This indicates that Ansible's become (*sudo*) worked properly, privilege escalation was successful, and remote commands were being executed as root user on the target server. This means that the *ansible_become_password* stored in Vault was applied correctly.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "whoami" --become --vault-password-file .secrets/vault_pass.txt
ubuntu-prod | CHANGED | rc=0 >>
root
ubuntu-dev | CHANGED | rc=0 >>
root
hasib@fedora-dev:~/devops-lab2-task3$
```

Full Playbook Run (site.yml) and Tags:

Syntax check:

In this step, pre-flight validation is performed before running the *site.yml* playbook using *--syntax-check*. This checks for YAML indentation or structure errors, role include or *vars_files* paths, and whether the Vault file is being accessed properly.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --syntax-check --vault-password-file .secrets/vault_pass.txt
```

Showing only the playbook name in the output means that no errors were found, meaning the syntax is correct and the playbook was parsed successfully.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --syntax-check --vault-password-file .secrets/vault_pass.txt

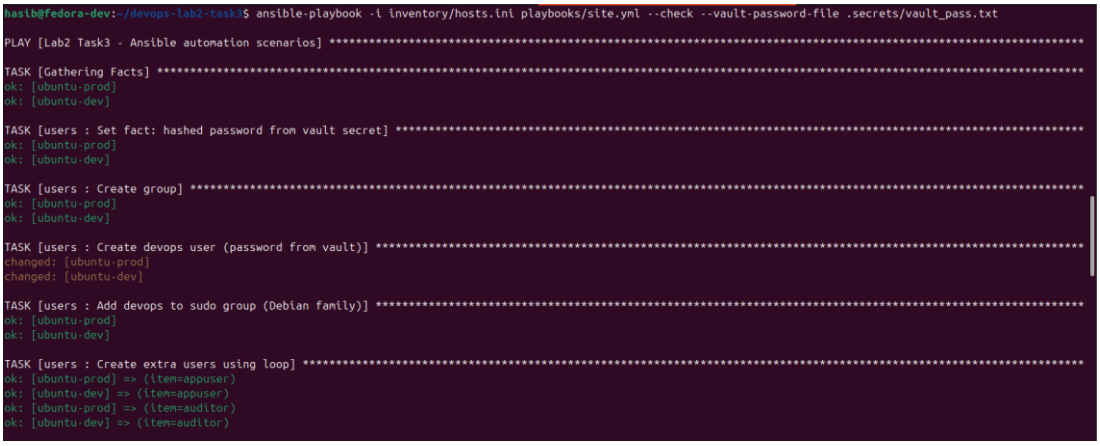
playbook: playbooks/site.yml
```

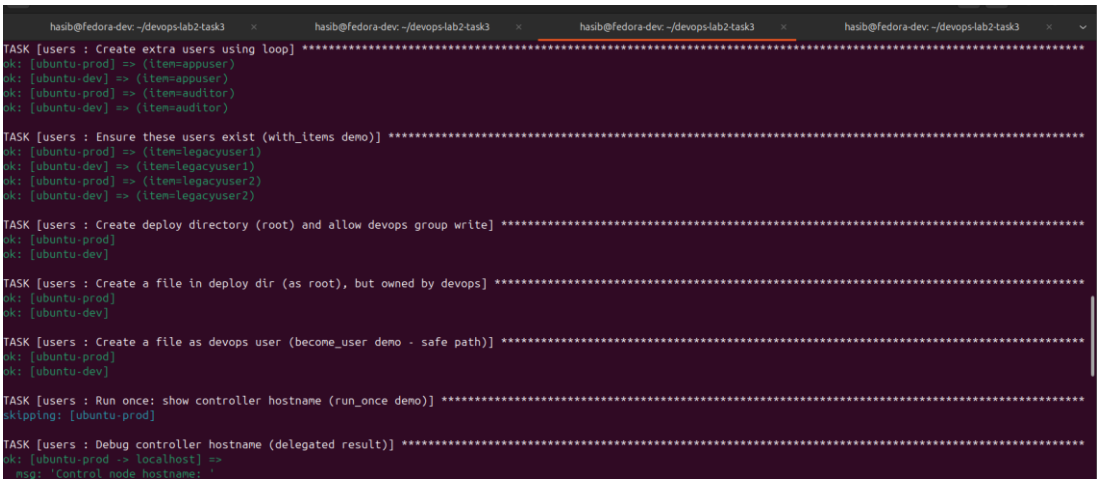
Check mode:

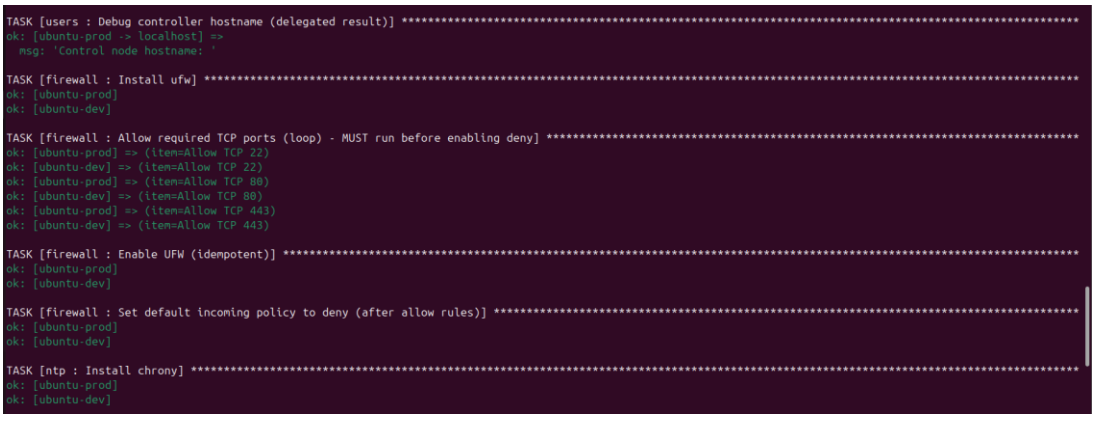
In this step, Check Mode (*--check*) is used to understand the changes that would occur if the playbook were run before actually applying them. This is very useful in a production-like environment, as it allows you to see the potential impact in advance and makes it clear which task changes will occur.

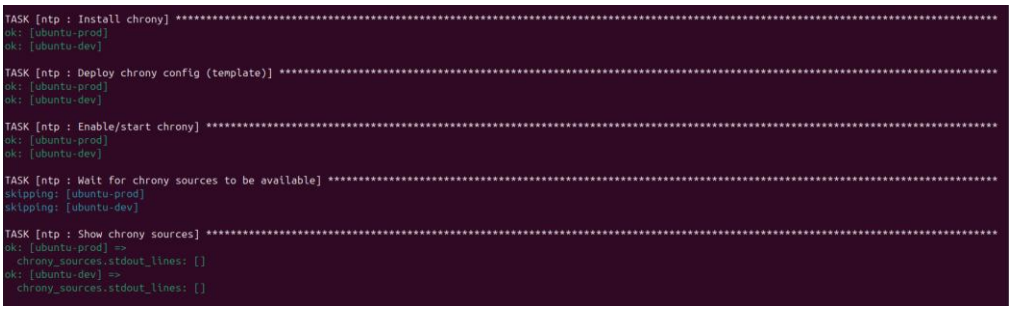
```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --check --vault-password-file .secrets/vault_pass.txt
```

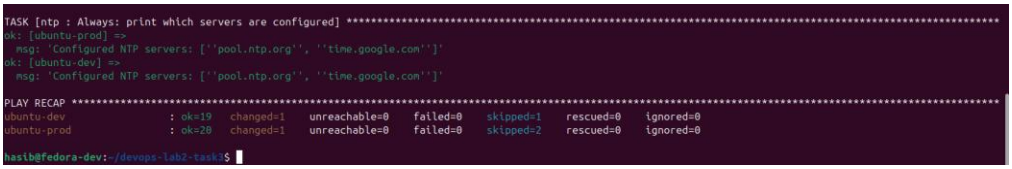
The output shows that both hosts in the inventory are reachable and facts have been collected correctly. The changed status for the devops user in the Users role means that a user create/update was required, which is expected, because the plan was to apply the hash from the password vault. The extra users (appuser, auditor) have been processed correctly through the loop, so the loop logic is correct.











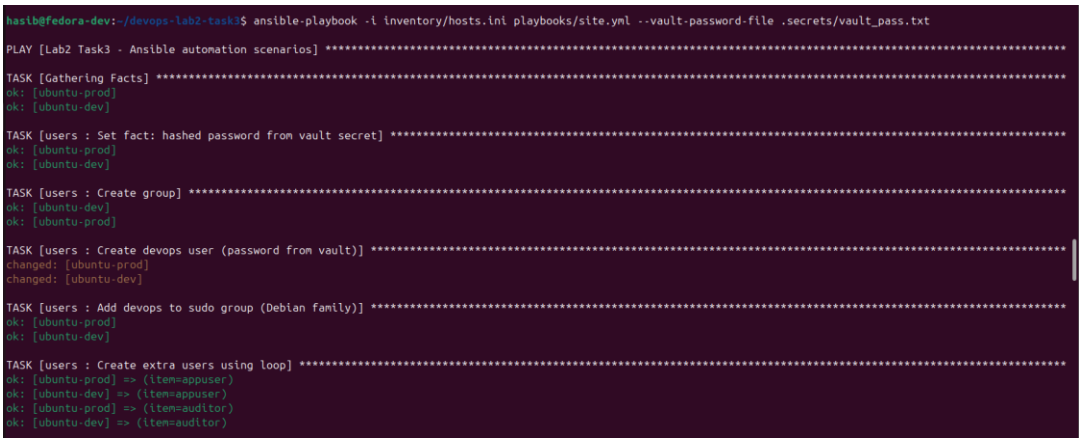
The hostname output for the *run_once* and *delegate_to* tasks is empty because some commands or delegated tasks in check mode do not give completely predictable output. This is a natural limitation of dry-run. For the same reason, the *chronyc* command in the NTP verify part is skipped because check mode runtime state-dependent verification cannot always be performed reliably.

Full run:

Full run is done because check mode only shows the plan, but this run actually applies the roles. According to the output, both hosts were reachable, no tasks failed, and the low changed count means that the playbook is quite idempotent.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --vault-password-file .secrets/vault_pass.txt
```

The *run_once* + *delegate_to* part shows evidence that the task was executed only once and since it was delegated to localhost, it ran on the control node (fedora-dev) instead of the target host, so the hostname correctly showed fedora-dev. Similarly, in the NTP advanced scenario, which was skipped in check mode, chronyc sources *-v* runs successfully in full run, meaning chrony has been installed, config deployed, service started and NTP sources have actually been caught.



```
TASK [users : Ensure these users exist (with_items demo)] ***********************************************************************************************
ok: [ubuntu-prod] => (item=legacyuser1)
ok: [ubuntu-dev] => (item=legacyuser1)
ok: [ubuntu-prod] => (item=legacyuser2)
ok: [ubuntu-dev] => (item=legacyuser2)

TASK [users : Create deploy directory (root) and allow devops group write] ******************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [users : Create a file in deploy dir (as root), but owned by devops] *******************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [users : Create a file as devops user (become_user demo - safe path)] ******************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [users : Run once: show controller hostname (run_once demo)] ***************************************************************************************
ok: [ubuntu-prod -> localhost]

TASK [users : Debug controller hostname (delegated result)] *********************************************************************************************
ok: [ubuntu-prod -> localhost] =>
  msg: 'Control node hostname: fedora-dev'

TASK [firewall : Install ufw] ***************************************************************************************************************************
ok: [ubuntu-dev]
ok: [ubuntu-prod]
```

```
TASK [firewall : Allow required TCP ports (loop) - MUST run before enabling deny] ***********************************************************************
ok: [ubuntu-dev] => (item=Allow TCP 22)
ok: [ubuntu-prod] => (item=Allow TCP 22)
ok: [ubuntu-dev] => (item=Allow TCP 80)
ok: [ubuntu-prod] => (item=Allow TCP 80)
ok: [ubuntu-dev] => (item=Allow TCP 443)
ok: [ubuntu-prod] => (item=Allow TCP 443)

TASK [firewall : Enable UFW (idempotent)] ***************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [firewall : Set default incoming policy to deny (after allow rules)] *******************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [ntp : Install chrony] *****************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [ntp : Deploy chrony config (template)] ************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [ntp : Enable/start chrony] ************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [ntp : Wait for chrony sources to be available] ****************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]
```

```
TASK [firewall : Enable UFW (idempotent)] ***************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [firewall : Set default incoming policy to deny (after allow rules)] *******************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [ntp : Install chrony] *****************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [ntp : Deploy chrony config (template)] ************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [ntp : Enable/start chrony] ************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [ntp : Wait for chrony sources to be available] ****************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]
```

```
TASK [ntp : Show chrony sources] ***********************************************************************************************************************************
ok: [ubuntu-prod] =>
  chrony_sources.stdout_lines:
  - ''
  - '  .-- Source mode  ''^'' = server, ''='' = peer, ''#'' = local clock.'
  - ' / .- Source state ''*'' = current best, ''+'' = combined, ''-'' = not combined,'
  - '| /             ''x'' = may be in error, ''~'' = too variable, ''?'' = unusable.'
  - '||                                                 .- xxxx [ yyyy ] +/- zzzz'
  - '||      Reachability register (octal) -.           |  xxxx = adjusted offset,'
  - '||      Log2(Polling interval) --.      |          |  yyyy = measured offset,'
  - '||                                \     |          |  zzzz = estimated error.'
  - '||                                 |    |          \'
  - 'MS Name/IP address         Stratum Poll Reach LastRx Last sample               '
  - ===============================================================================
  - ^- mail.rashnikov.name           2   7   377    49   +152us[ +152us] +/-   45ms
  - ^* 51.250.35.68                  2   8   377   179   +369us[  +60us] +/-   15ms
  - ^+ 82.202.138.88                 4   6   377    50   +533us[ +533us] +/-   28ms
  - ^+ 83.237.235.30                 2   8   377   115  -3890us[-3890us] +/-   29ms
  - ^+ time3.google.com              1   8   377   180   -582us[ -582us] +/-   21ms
  - ^+ time4.google.com              1   8   377    47   -835us[ -835us] +/-   20ms
  - ^+ time1.google.com              1   8   377   178  +2333us[+2333us] +/-   20ms
  - ^+ time2.google.com              1   8   377   178  -1257us[-1257us] +/-   21ms
ok: [ubuntu-dev] =>
  chrony_sources.stdout_lines:
  - ''
  - '  .-- Source mode  ''^'' = server, ''='' = peer, ''#'' = local clock.'
  - ' / .- Source state ''*'' = current best, ''+'' = combined, ''-'' = not combined,'
  - '| /             ''x'' = may be in error, ''~'' = too variable, ''?'' = unusable.'
  - '||                                                 .- xxxx [ yyyy ] +/- zzzz'
  - '||      Reachability register (octal) -.           |  xxxx = adjusted offset,'
  - '||      Log2(Polling interval) --.      |          |  yyyy = measured offset,'
```

```
  chrony_sources.stdout_lines:
  - ''
  - '  .-- Source mode  ''^'' = server, ''='' = peer, ''#'' = local clock.'
  - ' / .- Source state ''*'' = current best, ''+'' = combined, ''-'' = not combined,'
  - '| /             ''x'' = may be in error, ''~'' = too variable, ''?'' = unusable.'
  - '||                                                 .- xxxx [ yyyy ] +/- zzzz'
  - '||      Reachability register (octal) -.           |  xxxx = adjusted offset,'
  - '||      Log2(Polling interval) --.      |          |  yyyy = measured offset,'
  - '||                                \     |          |  zzzz = estimated error.'
  - '||                                 |    |          \'
  - 'MS Name/IP address         Stratum Poll Reach LastRx Last sample               '
  - ===============================================================================
  - ^* 51.250.35.68                  2   8   377   195    -98us[ -193us] +/-   15ms
  - ^+ 82.202.138.88                 4   7   377    68   +170us[ +170us] +/-   28ms
  - ^+ mail.rashnikov.name           2   8   377   138   -566us[ -566us] +/-   43ms
  - ^+ 83.237.235.30                 2   8   377    71  -4121us[-4121us] +/-   30ms
  - ^+ time1.google.com              1   8   377   195   -422us[ -422us] +/-   21ms
  - ^+ time3.google.com              1   8   377   129  -1446us[-1446us] +/-   20ms
  - ^+ time2.google.com              1   8   377   202  -2141us[-2235us] +/-   21ms
  - ^+ time4.google.com              1   8   377   194  +1275us[+1275us] +/-   21ms

TASK [ntp : Always: print which servers are configured] ************************************************************************************************************
ok: [ubuntu-prod] =>
  msg: 'Configured NTP servers: [''pool.ntp.org'', ''time.google.com'']'
ok: [ubuntu-dev] =>
  msg: 'Configured NTP servers: [''pool.ntp.org'', ''time.google.com'']'

PLAY RECAP *********************************************************************************************************************************************************
ubuntu-dev                 : ok=20   changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu-prod                : ok=22   changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

hasib@fedora-dev:~/devops-lab2-task3$
```

The output prod also shows pool.ntp.org, likely due to explicitly loading *group_vars/all/main.yml* with *vars_files in site.yml*. This can bypass the expected behavior of normal *group_vars* precedence. As a final refinement, relying on Ansible's automatic *group_vars* loading correctly reflects prod overrides (e.g. time.cloudflare.com).

Light Scenario

Users & Groups Management:

Users only run:

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --tags users --vault-password-file .secrets/vault_pass.txt
```

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --tags users --vault-password-file .secrets/vault_pass.txt

PLAY [Lab2 Task3 - Ansible automation scenarios] ********************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [users : Set fact: hashed password from vault secret] *********************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [users : Create group] ****************************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [users : Create devops user (password from vault)] ************************************************************************************************************
changed: [ubuntu-prod]
changed: [ubuntu-dev]

TASK [users : Add devops to sudo group (Debian family)] ************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [users : Create extra users using loop] ***********************************************************************************************************************
ok: [ubuntu-prod] => (item=appuser)
ok: [ubuntu-dev] => (item=appuser)
ok: [ubuntu-prod] => (item=auditor)
ok: [ubuntu-dev] => (item=auditor)
```

```
TASK [users : Create deploy directory (root) and allow devops group write] *****************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [users : Create a file in deploy dir (as root), but owned by devops] ******************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [users : Create a file as devops user (become_user demo - safe path)] *****************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [users : Run once: show controller hostname (run_once demo)] **************************************************************************************************
ok: [ubuntu-prod -> localhost]

TASK [users : Debug controller hostname (delegated result)] ********************************************************************************************************
ok: [ubuntu-prod -> localhost] =>
  msg: 'Control node hostname: fedora-dev'

PLAY RECAP *********************************************************************************************************************************************************
ubuntu-dev                 : ok=10   changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu-prod                : ok=12   changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Success verification (devops user exists):

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "id devops" --become --vault-password-file .secrets/vault_pass.txt
```

The output of id devops shows *uid=1001(devops*) and *gid=1001(devops)* which means that both devops user and devops group have been created successfully. At the same time, *groups=...,27(sudo)* proves that devops user has been added to *sudo group*. This means that the main purpose of users role has been achieved properly.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "id devops" --become --vault-password-file .secrets/vault_pass.txt
ubuntu-prod | CHANGED | rc=0 >>
uid=1001(devops) gid=1001(devops) groups=1001(devops),27(sudo)
ubuntu-dev | CHANGED | rc=0 >>
uid=1001(devops) gid=1001(devops) groups=1001(devops),27(sudo)
```

The output of the file */home/devops/become_user_demo.txt -rw-r--r-- 1 devops devops ...* shows that the file does indeed exist and the group is *devops:devops.*

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "ls -l /home/devops/become_user_demo.txt" --become --vault-password-file .se
crets/vault_pass.txt
```

It is clear that the task was executed in the *become_user:* devops context, not as root. As a result, the *become_user* demonstration was also successfully verified.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "ls -l /home/devops/become_user_demo.txt" --become --vault-password-file .se
crets/vault_pass.txt
ubuntu-prod | CHANGED | rc=0 >>
-rw-r--r-- 1 devops devops 47 Jan 28 19:35 /home/devops/become_user_demo.txt
ubuntu-dev | CHANGED | rc=0 >>
-rw-r--r-- 1 devops devops 47 Jan 28 19:35 /home/devops/become_user_demo.txt
hasib@fedora-dev:~/devops-lab2-task3$
```

Light Scenario

Firewall - UFW:

Firewall tag run:

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --tags firewall --vault-password-file .secrets/vault_pass.txt
```

Only the firewall role was run with *--tags* firewall, and the recap showing *changed=0* means that the rules were already in the correct state, where the playbook did not make unnecessary changes even after running it again. This is the idempotency proof of firewall automation.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --tags firewall --vault-password-file .secrets/vault_pass.txt

PLAY [Lab2 Task3 - Ansible automation scenarios] ********************************************************************************************************************

TASK [Gathering Facts] **********************************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [firewall : Install ufw] ***************************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [firewall : Allow required TCP ports (loop) - MUST run before enabling deny] ***********************************************************************************
ok: [ubuntu-prod] => (item=Allow TCP 22)
ok: [ubuntu-dev] => (item=Allow TCP 22)
ok: [ubuntu-prod] => (item=Allow TCP 80)
ok: [ubuntu-dev] => (item=Allow TCP 80)
ok: [ubuntu-prod] => (item=Allow TCP 443)
ok: [ubuntu-dev] => (item=Allow TCP 443)

TASK [firewall : Enable UFW (idempotent)] ***************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [firewall : Set default incoming policy to deny (after allow rules)] *******************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]
```

```
TASK [firewall : Set default incoming policy to deny (after allow rules)] *******************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

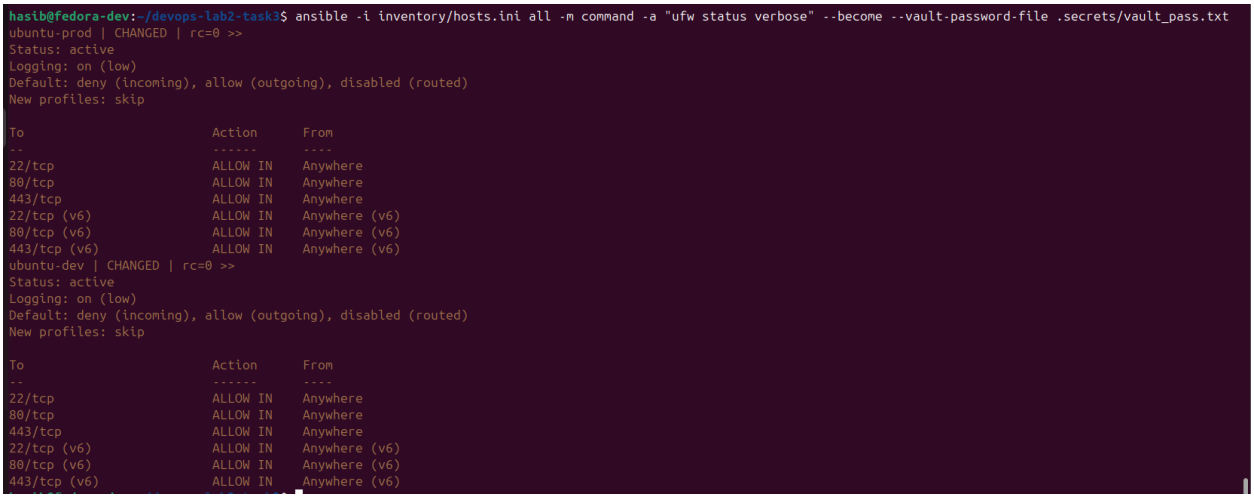
PLAY RECAP **********************************************************************************************************************************************************
ubuntu-dev                 : ok=5    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu-prod                : ok=5    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

hasib@fedora-dev:~/devops-lab2-task3$
```

Issue A: play "stuck":

I had to press *Ctrl+C* because it got stuck, which can sometimes be due to UFW enable/policy apply, temporary SSH latency, or remote session state mismatch - but that's what I did. After the interrupt, I verified the firewall state with *ufw status verbose* to see if it was OK. The output shows *Status: active*, Default: *deny (incoming)* and *22/80/443 allow*, which means that despite the interrupt, the final state was correct (it was already aligned). Then the status was checked.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "ufw status verbose" --become --vault-password-file .secrets/vault_pass.txt
```



Issue B: prod SSH timeout (port 22 blocked):

Port 22 timeout in ubuntu-prod is critical because if SSH (22/tcp) is closed/blocked, Ansible can no longer access prod. As a result, the prod host becomes “unreachable” and the entire automation pipeline breaks. Therefore, the most important safety point in firewall automation is to ensure SSH 22/tcp is allowed and then apply the deny/enable policy, which is the main reason for setting 22 in the role variable allowed_tcp_ports.

Manual Fixing on prod host:

In this step, manual intervention was performed by going directly to the ubuntu-prod VM, because SSH port 22/tcp was blocked and it was no longer possible to connect from the control node.

```
hasib@ubuntu-prod:~$ sudo ufw status verbose
```

The current firewall status is displayed with *ufw status verbose*.

```
hasib@ubuntu-prod:~$ sudo ufw status verbose
[sudo] password for hasib:
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip
```

Then SSH access was opened again with *ufw allow 22/tcp* and finally ufw was reloaded and the rule was applied.

```
hasib@ubuntu-prod:~$ sudo ufw allow 22/tcp
sudo ufw reload
```

The output showing Rule added and Firewall reloaded means that the SSH port has been successfully re-allowed and the prod VM is reachable again. This allows the production host to recover.

```
hasib@ubuntu-prod:~$ sudo ufw allow 22/tcp
sudo ufw reload
Rule added
Rule added (v6)
Firewall reloaded
```

Connectivity check from control node:

After the fix, connectivity proof was taken from the control node to ensure that the prod host was fully recovered.

```
hasib@fedora-dev:~/devops-lab2-task3$ ping -c 3 192.168.145.129
```

ICMP ping found 0% packet loss, which indicates network reachability is OK.

```
hasib@fedora-dev:~/devops-lab2-task3$ ping -c 3 192.168.145.129
PING 192.168.145.129 (192.168.145.129) 56(84) bytes of data.
64 bytes from 192.168.145.129: icmp_seq=1 ttl=64 time=4.08 ms
64 bytes from 192.168.145.129: icmp_seq=2 ttl=64 time=1.46 ms
64 bytes from 192.168.145.129: icmp_seq=3 ttl=64 time=1.11 ms

--- 192.168.145.129 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.108/2.215/4.075/1.323 ms
```

Then check *port 22/tcp* with *nc -vz* and get succeeded, which means the SSH port is open.

```
hasib@fedora-dev:~/devops-lab2-task3$ nc -vz 192.168.145.129 22
```

```
hasib@fedora-dev:~/devops-lab2-task3$ nc -vz 192.168.145.129 22
Connection to 192.168.145.129 22 port [tcp/ssh] succeeded!
```

Finally, running ansible *-m ping* in verbose mode yielded *SUCCESS => pong*, which proves that Ansible is able to connect to the prod host again properly.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini prod -m ping --vault-password-file .secrets/vault_pass.txt -vv
```

Combining these three tests, it is clear that SSH connectivity has been fully restored.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini prod -m ping --vault-password-file .secrets/vault_pass.txt -vv
ansible [core 2.16.3]
  config file = /home/hasib/devops-lab2-task3/ansible.cfg
  configured module search path = ['/home/hasib/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/hasib/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Jan  8 2026, 11:30:50) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
Using /home/hasib/devops-lab2-task3/ansible.cfg as config file
Skipping callback 'default', as we already have a stdout callback.
Skipping callback 'minimal', as we already have a stdout callback.
Skipping callback 'oneline', as we already have a stdout callback.
ubuntu-prod | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

Advanced Scenario

NTP (Chrony):

NTP tag run:

Only the NTP role was run with *--tags ntp*, so that the Chrony setup could be verified separately. All tasks ok in the output and *changed=0* for both hosts in the PLAY RECAP means that Chrony was already in the expected state. Rerunning the playbook did not make any unnecessary changes. This proves the idempotence of the role.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --tags ntp --vault-password-file .secrets/vault_pass.txt
```

It is important to see ^* in the chronyc sources *-v* output, because in Chrony, ^* means the current best time source. This means that the system time is actually syncing from the NTP server and Chrony is working properly.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --tags ntp --vault-password-file .secrets/vault_pass.txt

PLAY [Lab2 Task3 - Ansible automation scenarios] ********************************************************************************************************************

TASK [Gathering Facts] **********************************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [ntp : Install chrony] *****************************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [ntp : Deploy chrony config (template)] ************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [ntp : Enable/start chrony] ************************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [ntp : Wait for chrony sources to be available] ****************************************************************************************************************
ok: [ubuntu-dev]
ok: [ubuntu-prod]
```

```
TASK [ntp : Show chrony sources] ***********************************************************************************************************************************
ok: [ubuntu-prod] =>
  chrony_sources.stdout_lines:
  - ''
  - '  .-- Source mode  ''^'' = server, ''='' = peer, ''#'' = local clock.'
  - ' / .- Source state ''*'' = current best, ''+'' = combined, ''-'' = not combined,'
  - '| /             ''x'' = may be in error, ''~'' = too variable, ''?'' = unusable.'
  - '||                                                 .- xxxx [ yyyy ] +/- zzzz'
  - '||      Reachability register (octal) -.           |  xxxx = adjusted offset,'
  - '||      Log2(Polling interval) --.      |          |  yyyy = measured offset,'
  - '||                                \     |          |  zzzz = estimated error.'
  - '||                                 |    |          \'
  - 'MS Name/IP address         Stratum Poll Reach LastRx Last sample               '
  - ===============================================================================
  - ^- mail.rashnikov.name          2   9   377    56  +1033us[+1033us] +/-   41ms
  - ^* 51.250.35.68                 2   9   377   195  +1193us[+1230us] +/-   15ms
  - ^+ 82.202.138.88                4   8   377   126  -4109us[-4109us] +/-   25ms
  - ^+ 83.237.235.30                2   9   377   142  -3465us[-3465us] +/-   31ms
  - ^+ time3.google.com             1  10   377   693  +3111us[+3166us] +/-   20ms
  - ^+ time4.google.com             1  10   377   581   +692us[ +742us] +/-   23ms
  - ^+ time1.google.com             1  10   377   708   -152us[  -97us] +/-   21ms
  - ^+ time2.google.com             1  10   377   183  +2094us[+2094us] +/-   21ms
ok: [ubuntu-dev] =>
```

```
  chrony_sources.stdout_lines:
  - ''
  - '  .-- Source mode  ''^'' = server, ''='' = peer, ''#'' = local clock.'
  - ' / .- Source state ''*'' = current best, ''+'' = combined, ''-'' = not combined,'
  - '| /             ''x'' = may be in error, ''~'' = too variable, ''?'' = unusable.'
  - '||                                                 .- xxxx [ yyyy ] +/- zzzz'
  - '||      Reachability register (octal) -.           |  xxxx = adjusted offset,'
  - '||      Log2(Polling interval) --.      |          |  yyyy = measured offset,'
  - '||                                \     |          |  zzzz = estimated error.'
  - '||                                 |    |          \'
  - 'MS Name/IP address         Stratum Poll Reach LastRx Last sample               '
  - ===============================================================================
  - ^* 51.250.35.68                 2   9   377   466   +286us[ +441us] +/-   15ms
  - ^+ 82.202.138.88                4   7   377    78  -5438us[-5438us] +/-   25ms
  - ^- mail.rashnikov.name          2   9   377   143   +369us[ +369us] +/-   40ms
  - ^+ 83.237.235.30                2  10   377    89  -4129us[-4129us] +/-   29ms
  - ^+ time1.google.com             1  10   377   713  +2841us[+2990us] +/-   20ms
  - ^+ time3.google.com             1  10   377   648  +1297us[+1447us] +/-   19ms
  - ^+ time2.google.com             1  10   377   726  +1573us[+1721us] +/-   21ms
  - ^+ time4.google.com             1  10   377   706   -157us[-8076ns] +/-   23ms

TASK [ntp : Always: print which servers are configured] ************************************************************************************************************
ok: [ubuntu-prod] =>
  msg: 'Configured NTP servers: [''pool.ntp.org'', ''time.google.com'']'
ok: [ubuntu-dev] =>
  msg: 'Configured NTP servers: [''pool.ntp.org'', ''time.google.com'']'

PLAY RECAP *********************************************************************************************************************************************************
ubuntu-dev                 : ok=7    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu-prod                : ok=7    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Verify:

This verification is very strong as a command-based “hard proof”. Ansible ... "*chronyc sources -v | head -n 15*" output from both hosts. chrony is running properly and the chronyc command was executed.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "chronyc sources -v | head -n 15" --become --vault-password-file .secrets/va
ult_pass.txt
```

The presence of a ^* line in the output means that Chrony has picked a selected time source, meaning that the sources are reachable and the system time is actually synchronized. This is proof that the chrony service is running, NTP sources are accessible, and time sync is active.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "chronyc sources -v | head -n 15" --become --vault-password-file .secrets/va
ult_pass.txt
ubuntu-prod | CHANGED | rc=0 >>

  .-- Source mode  '^' = server, '=' = peer, '#' = local clock.
 / .- Source state '*' = current best, '+' = combined, '-' = not combined,
| /             'x' = may be in error, '~' = too variable, '?' = unusable.
||                                                 .- xxxx [ yyyy ] +/- zzzz
||      Reachability register (octal) -.           |  xxxx = adjusted offset,
||      Log2(Polling interval) --.      |          |  yyyy = measured offset,
||                                \     |          |  zzzz = estimated error.
||                                 |    |          \
MS Name/IP address         Stratum Poll Reach LastRx Last sample
===============================================================================
^- mail.rashnikov.name          2   9   377   422  +1570us[+1033us] +/-   41ms
^* 51.250.35.68                 2   9   377    38  +1047us[ +502us] +/-   15ms
^+ 82.202.138.88                4   8   377   232  -6112us[-6652us] +/-   26ms
^+ 83.237.235.30                2   9   377   508  -2930us[-3465us] +/-   31ms
^+ time3.google.com             1  10   377    19  +1889us[+1889us] +/-   22ms
^+ time4.google.com             1  10   377   947  +1217us[ +742us] +/-   23ms
^+ time1.google.com             1  10   377    38  +1875us[+1330us] +/-   19ms
^+ time2.google.com             1  10   377   548  +2628us[+2094us] +/-   21ms
ubuntu-dev | CHANGED | rc=0 >>
```

```
  .-- Source mode  '^' = server, '=' = peer, '#' = local clock.
 / .- Source state '*' = current best, '+' = combined, '-' = not combined,
| /             'x' = may be in error, '~' = too variable, '?' = unusable.
||                                                 .- xxxx [ yyyy ] +/- zzzz
||      Reachability register (octal) -.           |  xxxx = adjusted offset,
||      Log2(Polling interval) --.      |          |  yyyy = measured offset,
||                                \     |          |  zzzz = estimated error.
||                                 |    |           \
MS Name/IP address         Stratum Poll Reach LastRx Last sample
===============================================================================
^* 51.250.35.68                  2   9   377   318   +926us[ +291us] +/-   14ms
^+ 82.202.138.88                 4   7   377    54  -3917us[-3917us] +/-   28ms
^- mail.rashnikov.name           2   9   377   509  +1000us[ +369us] +/-   40ms
^+ 83.237.235.30                 2  10   377   455  -3497us[-4129us] +/-   29ms
^+ time1.google.com              1  10   377    39   -346us[ -346us] +/-   21ms
^+ time3.google.com              1  10   377  1014  +1918us[+1447us] +/-   19ms
^+ time2.google.com              1  10   377    50   +381us[ +381us] +/-   23ms
^+ time4.google.com              1  10   377    36  -1138us[-1138us] +/-   22ms
hasib@fedora-dev:~/devops-lab2-task3$
```

Bonus

Ansible-Lint:

PEP 668 “externally-managed-environment” (pip install blocked):

According to the new Python packaging policy (*PEP 668*), many distros block *pip install* by treating the system Python environment as “externally managed” to avoid conflicts with OS-managed packages. So, when installing tools in the system environment using pip, you may get stuck with an “externally-managed-environment” error. So, as per best practice, ansible-lint is installed using the apt package manager. It is system-managed, safe, and compatible with the distro.

```
hasib@fedora-dev:~/devops-lab2-task3$ sudo apt install ansible-lint
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  black git git-man liberror-perl python3-ansible-compat python3-bracex python3-filelock python3-mypy-extensions python3-pathspec python3-ruamel.yaml
  python3-ruamel.yaml.clib python3-subprocess-tee python3-wcmatch yamllint
Suggested packages:
  python-black-doc git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn
The following NEW packages will be installed:
  ansible-lint black git git-man liberror-perl python3-ansible-compat python3-bracex python3-filelock python3-mypy-extensions python3-pathspec
  python3-ruamel.yaml python3-ruamel.yaml.clib python3-subprocess-tee python3-wcmatch yamllint
0 upgraded, 15 newly installed, 0 to remove and 148 not upgraded.
Need to get 5,486 kB of archives.
After this operation, 27.8 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 python3-mypy-extensions all 1.0.0-1 [6,148 B]
Get:2 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 python3-pathspec all 0.12.1-1 [24.5 kB]
Get:3 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 black all 24.2.0-1ubuntu1 [140 kB]
Get:4 http://ru.archive.ubuntu.com/ubuntu noble/main amd64 liberror-perl all 0.17029-2 [25.6 kB]
Get:5 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 git-man all 1:2.43.0-1ubuntu7.3 [1,100 kB]
Get:6 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 git amd64 1:2.43.0-1ubuntu7.3 [3,680 kB]
Get:7 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 python3-subprocess-tee all 0.4.1-1 [6,238 B]
Get:8 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 python3-ansible-compat all 4.1.11-1 [20.8 kB]
Get:9 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 python3-filelock all 3.13.1-1 [10.8 kB]
Get:10 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 python3-ruamel.yaml.clib amd64 0.2.8-1build1 [132 kB]
Get:11 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 python3-ruamel.yaml all 0.17.21-1 [86.6 kB]
Get:12 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 python3-bracex all 2.2.1-2 [10.5 kB]
Get:13 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 python3-wcmatch all 8.4-2ubuntu0.1 [32.8 kB]
Get:14 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 yamllint all 1.33.0-1 [47.2 kB]
Get:15 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 ansible-lint all 6.17.2-1 [163 kB]
```

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-lint --version
ansible-lint 6.17.2 using ansible-core:2.16.3 ansible-compat:4.1.11 ruamel-yaml:0.17.21 ruamel-yaml-clib:0.2.8
```

The output shows *ansible-lint 6.17.2* using *ansible-core:2.16.3 ...* , which proves that the tool is installed correctly and is compatible with the ansible-core version used.

First ansible-lint crash (PermissionError: /root/galaxy.yml):

```
hasib@fedora-dev:~/devops-lab2-task3$ ~/.local/bin/ansible-lint playbooks/site.yml
```

This crash occurred because ansible-lint (*ansible-compat* inside) incorrectly resolved the *cache/home* path while preparing the runtime. The warning shows that it assumed */.ansible* as the caching directory - it is in the root filesystem, so the user cannot write there; that is why the "not writable" warning was given. Due to the same incorrect resolve, the tool checked */root/galaxy.yml* while scanning the galaxy path, but it returned *PermissionError: ... '/root/galaxy.yml'* because */root* is restricted for normal users.

```
hasib@fedora-dev:~/devops-lab2-task3$ ~/.local/bin/ansible-lint playbooks/site.yml
WARNING  Project directory /.ansible cannot be used for caching as it is not writable.
WARNING  Using unique temporary directory /tmp/.ansible-0aaa for caching.
/home/hasib/.local/share/pipx/venvs/ansible-lint/lib/python3.12/site-packages/ansible_compat/runtime.py:242: UserWarning: Project directory /.ansible cannot be used for caching as it is not writable.
  self.cache_dir = get_cache_dir(self.project_dir, isolated=self.isolated)
/home/hasib/.local/share/pipx/venvs/ansible-lint/lib/python3.12/site-packages/ansible_compat/runtime.py:242: UserWarning: Using unique temporary directory /tmp/.ansible-0aaa for caching.
  self.cache_dir = get_cache_dir(self.project_dir, isolated=self.isolated)
Traceback (most recent call last):
  File "/home/hasib/.local/bin/ansible-lint", line 7, in <module>
    sys.exit(_run_cli_entrypoint())
             ^^^^^^^^^^^^^^^^^^^^
  File "/home/hasib/.local/share/pipx/venvs/ansible-lint/lib/python3.12/site-packages/ansiblelint/__main__.py", line 447, in _run_cli_entrypoint
    sys.exit(main(sys.argv))
             ^^^^^^^^^^^^^^
  File "/home/hasib/.local/share/pipx/venvs/ansible-lint/lib/python3.12/site-packages/ansiblelint/__main__.py", line 376, in main
    app = get_app(
          ^^^^^^^^
  File "/home/hasib/.local/share/pipx/venvs/ansible-lint/lib/python3.12/site-packages/ansiblelint/app.py", line 434, in get_app
    app.runtime.prepare_environment(
  File "/home/hasib/.local/share/pipx/venvs/ansible-lint/lib/python3.12/site-packages/ansible_compat/runtime.py", line 741, in prepare_environment
    for item in search_galaxy_paths(self.project_dir):
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/hasib/.local/share/pipx/venvs/ansible-lint/lib/python3.12/site-packages/ansible_compat/runtime.py", line 1075, in search_galaxy_paths
    if file_path.exists():
       ^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/pathlib.py", line 862, in exists
    self.stat(follow_symlinks=follow_symlinks)
  File "/usr/lib/python3.12/pathlib.py", line 842, in stat
```

fix steps:

```
hasib@fedora-dev:~/devops-lab2-task3$ whoami
hasib
```

Confirmed with *whoami*, *echo "$HOME"* showed HOME is correctly set to */home/hasib*.

```
hasib@fedora-dev:~/devops-lab2-task3$ echo "$HOME"
/home/hasib
```

Confirmed from *pwd* that the playbook is running from the project directory (*/home/hasib/devops-lab2-task3*).

```
hasib@fedora-dev:~/devops-lab2-task3$ pwd
/home/hasib/devops-lab2-task3
```

Then, writable cache and temp directories were created, because ansible-lint and Ansible runtime write cache and temporary files during operation. So, safe paths in user-space were ensured by creating *~/.ansible/tmp and ~/.cache/ansible-lint*.

```
hasib@fedora-dev:~/devops-lab2-task3$ mkdir -p ~/.ansible/tmp ~/.cache/ansible-lint
```

Next, the environment variables are set so that the runtime is fully bound to user-space. HOME specifies the user home, ANSIBLE_HOME places all *.ansible*-related runtime data in the user directory, *XDG_CACHE_HOME* sets the cache path, and *ANSIBLE_LOCAL_TEMP* moves temporary files to a permission-safe location.

```
hasib@fedora-dev:~/devops-lab2-task3$ export HOME="/home/hasib"
```

```
hasib@fedora-dev:~/devops-lab2-task3$ export ANSIBLE_HOME="$HOME/.ansible"
```

```
hasib@fedora-dev:~/devops-lab2-task3$ export XDG_CACHE_HOME="$HOME/.cache"
```

```
hasib@fedora-dev:~/devops-lab2-task3$ export ANSIBLE_LOCAL_TEMP="$HOME/.ansible/tmp"
```

Additionally, the use of the project's own *ansible.cfg* is ensured by setting *ANSIBLE_CONFIG*.

```
hasib@fedora-dev:~/devops-lab2-task3$ export ANSIBLE_CONFIG="$PWD/ansible.cfg"
```

With these settings, ansible-lint no longer points to */root/galaxy.yml* or */.ansible*; everything will operate properly in user-space and the permission-related crash will be resolved.

ansible-lint internal-error:

Attempting to decrypt but no vault secrets found:

This error occurred because while parsing *playbooks/site.yml*, ansible-lint tried to decrypt *group_vars/all/vault.yml* in vars_files. No vault secret source (password/file) was requested during the lint run, so the tool could not decrypt and displayed Attempting to decrypt but no vault secrets found.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-lint --project-dir "$PWD" playbooks/site.yml
Invalid configuration file /home/hasib/devops-lab2-task3/.ansible-lint. Additional properties are not allowed ('config' was unexpected)
```

Fixing:

This fix worked because by setting the *ANSIBLE_VAULT_PASSWORD_FILE* environment variable, ansible-lint and the Ansible runtime were directly told where the key to unlock the Vault was. As a result, when the playbook needed to decrypt *vault.yml*, the tool automatically used that password file and no longer got the decryption error.

```
hasib@fedora-dev:~/devops-lab2-task3$ export ANSIBLE_VAULT_PASSWORD_FILE="$PWD/.secrets/vault_pass.txt"
```

Sometimes the *.ansible-lint cache/settings* file can retain the old state and create the same issue again, so it has been reset.

```
hasib@fedora-dev:~/devops-lab2-task3$ rm -f .ansible-lint
```

This time after running *ansible-lint* again, success came - *Passed: 0 failure(s), 0 warning(s).*

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-lint --project-dir "$PWD" playbooks/site.yml
```

This means the Vault file was successfully decrypted, passed all lint rules cleanly, and satisfied the lint profile up to “production”.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-lint --project-dir "$PWD" playbooks/site.yml

Passed: 0 failure(s), 0 warning(s) on 11 files. Last profile that met the validation criteria was 'production'.
hasib@fedora-dev:~/devops-lab2-task3$
```

Lint violations fixing:

Adding vault_password_file to ansible.cfg automatically:

This fix is made so that the vault password file is permanently set in the project-level configuration and does not have to be set separately on the command line. Ensuring *vault_password_file = .secrets/vault_pass.txt* in *ansible.cfg* ensures that all Ansible tools such as *ansible-playbook*, ansible CLI and even *ansible-lint* follow the same config.

```
hasib@fedora-dev:~/devops-lab2-task3$ grep -q '^vault_password_file' ansible.cfg || sed -i '/^roles_path =/a vault_password_file = .secrets/vault_pass.txt' ansible
.cfg
```

Removing extra blank lines from specific files (awk loop):

The fix below was made because ansible-lint can detect multiple consecutive blank lines in some YAML files as a format violation. Therefore, blank lines have been normalized in certain files.

The given awk loop does not change the actual content of the lines, but only reduces them to one where there are multiple blank lines. As a result, the YAML structure is correct, readability is maintained, and lint's format rules are passed.

```
hasib@fedora-dev:~/devops-lab2-task3$ for f in playbooks/site.yml \
  roles/firewall/handlers/main.yml \
  roles/firewall/tasks/main.yml \
  roles/users/tasks/create_users.yml \
  roles/users/tasks/demo_features.yml
do
  awk 'BEGIN{blank=0} /^[[:space:]]*$/{blank++; if(blank<=1) print; next} {blank=0; print}' "$f" > "$f.tmp" && mv "$f.tmp" "$f"
done
```

Handler names casing fixes (sed):

```
hasib@fedora-dev:~/devops-lab2-task3$ sed -i 's/^- name: *reload ufw$/- name: Reload UFW/' roles/firewall/handlers/main.yml
```

This fix is to keep the casing/format of notify and handler names consistent. In Ansible, notify: must be an exact match to the handler's *-name:* - if the case or spacing is different, errors or lint issues can occur.

Here, the handler names are made standard and consistent using sed (*Reload UFW*, *Restart chrony*). This makes the notify - handler mapping clearer, avoids runtime errors, and keeps both lint/production quality correct.

```
hasib@fedora-dev:~/devops-lab2-task3$ sed -i 's/^- name: *restart chrony$/- name: Restart chrony/' roles/ntp/handlers/main.yml
```

Import tasks best practice (FQCN + named tasks) - users role:

This rewrite is done perfectly to make it ansible-lint compliant. *roles/users/tasks/main.yml* is now clean and has a separate named task for each *import_tasks*, using FQCN (*ansible.builtin.import_tasks*). As a result, all three readability, maintainability and lint standards are satisfied.

Rewriting Users role main.yml:

```
hasib@fedora-dev:~/devops-lab2-task3$ cat > roles/users/tasks/main.yml <<'YAML'
---
- name: Import user creation tasks
  ansible.builtin.import_tasks: create_users.yml

- name: Import demo feature tasks
  ansible.builtin.import_tasks: demo_features.yml
YAML
```

Rewriting NTP role main.yml:

This rewrite is also done correctly according to ansible-lint best practices. *roles/ntp/tasks/main.yml* is now clean and readable, there are clearly named tasks for each *import_tasks* and FQCN (*ansible.builtin.import_tasks*) is used. This ensures lint compliance, and the role structure is clean.

```
hasib@fedora-dev:~/devops-lab2-task3$ cat > roles/ntp/tasks/main.yml <<'YAML'
---
- name: Import chrony install tasks
  ansible.builtin.import_tasks: install.yml

- name: Import chrony configure tasks
  ansible.builtin.import_tasks: configure.yml

- name: Import chrony verify tasks
  ansible.builtin.import_tasks: verify.yml
YAML
```

Replacing ignore_errors with failed_when/changed_when (perl)

lint usually tells you to avoid *ignore_errors: true* (because it hides silent failures). Instead, controlled failure handling is shown.

```
hasib@fedora-dev:~/devops-lab2-task3$ perl -0777 -i -pe 's/ignore_errors:\s*true\s*\n/failed_when: false\nchanged_when: false\n/gm' roles/firewall/handlers/main.ym
l
```

```
hasib@fedora-dev:~/devops-lab2-task3$ perl -0777 -i -pe 's/ignore_errors:\s*true\s*\n/failed_when: false\nchanged_when: false\n/gm' roles/ntp/tasks/verify.yml
```

Fixing YAML broken structure (rewriting firewall handler)

The handlers file has been rewritten to be clean and lint-friendly:

```
hasib@fedora-dev:~/devops-lab2-task3$ cat > roles/firewall/handlers/main.yml <<'YAML'
---
- name: Reload UFW
  ansible.builtin.command: ufw reload
  register: ufw_reload
  changed_when: false
  failed_when: false
YAML
```

verify.yml YAML error debug (line numbers):

Stray changed_*when: false* lines in *verify.yml* broke the structure. So debugged with line-number.

```
hasib@fedora-dev:~/devops-lab2-task3$ nl -ba roles/ntp/tasks/verify.yml | sed -n '1,80p'
```

```
hasib@fedora-dev:~/devops-lab2-task3$ nl -ba roles/ntp/tasks/verify.yml | sed -n '1,80p'
     1  ---
     2  - name: "Verify NTP sources (block/rescue demo)"
     3    block:
     4      - name: "Wait for chrony sources to be available"
     5        ansible.builtin.command: chronyc sources -v
     6        register: chrony_sources
     7        retries: 5
     8        delay: 3
     9        until: chrony_sources.rc == 0
    10        changed_when: false
    11
    12      - name: "Show chrony sources"
    13        ansible.builtin.debug:
    14          var: chrony_sources.stdout_lines
    15
    16    rescue:
    17      - name: "Chrony verification failed - show service status (ignore_errors demo)"
    18        ansible.builtin.command: systemctl status chrony --no-pager
    19        register: chrony_status
    20        changed_when: false
    21        failed_when: false
```

```
    16    rescue:
    17      - name: "Chrony verification failed - show service status (ignore_errors demo)"
    18        ansible.builtin.command: systemctl status chrony --no-pager
    19        register: chrony_status
    20        changed_when: false
    21        failed_when: false
    22  changed_when: false
    23      - name: "Debug chrony status"
    24        ansible.builtin.debug:
    25          var: chrony_status.stdout_lines
    26        failed_when: false
    27  changed_when: false
    28    always:
    29      - name: "Always: print which servers are configured"
    30        ansible.builtin.debug:
    31          msg: "Configured NTP servers: {{ ntp_servers }}"
```

Removing stray changed_when: false lines (sed):

Then the stray lines are removed:

```
hasib@fedora-dev:~/devops-lab2-task3$ sed -i '/^[[:space:]]*changed_when: false$/{
/^changed_when: false$/d
}' roles/ntp/tasks/verify.yml
```

Verifying file after fix:

Confirm again with line-number:

structure is now clean: block, rescue, always which is nested properly.

```
hasib@fedora-dev:~/devops-lab2-task3$ nl -ba roles/ntp/tasks/verify.yml | sed -n '1,80p'
```

```
hasib@fedora-dev:~/devops-lab2-task3$ nl -ba roles/ntp/tasks/verify.yml | sed -n '1,80p'
     1  ---
     2  - name: "Verify NTP sources (block/rescue demo)"
     3    block:
     4      - name: "Wait for chrony sources to be available"
     5        ansible.builtin.command: chronyc sources -v
     6        register: chrony_sources
     7        retries: 5
     8        delay: 3
     9        until: chrony_sources.rc == 0
    10        changed_when: false
    11
    12      - name: "Show chrony sources"
    13        ansible.builtin.debug:
    14          var: chrony_sources.stdout_lines
    15
    16    rescue:
    17      - name: "Chrony verification failed - show service status (ignore_errors demo)"
    18        ansible.builtin.command: systemctl status chrony --no-pager
    19        register: chrony_status
    20        changed_when: false
    21        failed_when: false
    22      - name: "Debug chrony status"
    23        ansible.builtin.debug:
    24          var: chrony_status.stdout_lines
    25        failed_when: false
    26    always:
    27      - name: "Always: print which servers are configured"
    28        ansible.builtin.debug:
    29          msg: "Configured NTP servers: {{ ntp_servers }}"
```

Checking syntax again:

After the lint fix, the playbook syntax was checked and confirmed:

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --syntax-check \
  --vault-password-file .secrets/vault_pass.txt

playbook: playbooks/site.yml
```

Final lint pass:

Final run (explicit env):

This final run is lint completion proof. Here, ansible-lint is run with *ANSIBLE_CONFIG* and *ANSIBLE_VAULT_PASSWORD_FILE* explicitly set, so that the tool uses the correct project config and Vault secret. 0 failures, 0 warnings in the output means that all lint rules have passed and production profile has been satisfied.

```
hasib@fedora-dev:~/devops-lab2-task3$ ANSIBLE_CONFIG="$PWD/ansible.cfg" \
ANSIBLE_VAULT_PASSWORD_FILE="$PWD/.secrets/vault_pass.txt" \
ansible-lint --project-dir "$PWD" playbooks/site.yml

Passed: 0 failure(s), 0 warning(s) on 11 files. Last profile that met the validation criteria was 'production'.
```

Blank lines verification:

This step is done as a final formatting proof, to show that there are no unnecessary blank lines in the YAML files. Each specific file is searched for empty lines using grep.

This means:

- All YAML files are clean and normalized
- No formatting/style violations
- Compliant with ansible-lint rules

```
hasib@fedora-dev:~/devops-lab2-task3$ for f in playbooks/site.yml roles/firewall/tasks/main.yml roles/users/tasks/create_users.yml roles/users/tasks/demo_features.
yml
do
  echo "== $f =="
  grep -n '^[[:space:]]*$' "$f" || echo "No blank lines "
done
```

No blank lines appears everywhere in the output.

```
hasib@fedora-dev:~/devops-lab2-task3$ for f in playbooks/site.yml roles/firewall/tasks/main.yml roles/users/tasks/create_users.yml roles/users/tasks/demo_features.
yml
do
  echo "== $f =="
  grep -n '^[[:space:]]*$' "$f" || echo "No blank lines "
done

== playbooks/site.yml ==
No blank lines
== roles/firewall/tasks/main.yml ==
No blank lines
== roles/users/tasks/create_users.yml ==
No blank lines
== roles/users/tasks/demo_features.yml ==
No blank lines
```

Final Verification:

In this part, all the requirements for the final submission - Users (Light), Firewall (Light), NTP/Chrony (Advanced) - have been applied correctly on both hosts (ubuntu-prod + ubuntu-dev).

Firewall status:

In this verification step, the actual status of the firewall is confirmed by running *ufw status verbose* on the host. *Status: active* in the output means that the firewall is on, *Default: deny (incoming)* means that all incoming traffic is blocked by default, and only *22/tcp (SSH), 80/tcp (HTTP), 443/tcp (HTTPS)* are allowed, which is the same rule for IPv6.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "ufw status verbose" --become --vault-password-file .secrets/vault_pass.txt
```

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "ufw status verbose" --become --vault-password-file .secrets/vault_pass.txt
ubuntu-prod | CHANGED | rc=0 >>
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)
ubuntu-dev | CHANGED | rc=0 >>
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)
hasib@fedora-dev:~/devops-lab2-task3$
```

It can be seen that the firewall policy has been properly enforced and all incoming ports except *SSH/HTTP/HTTPS* are closed as per the requirement. This means that the production-grade firewall configuration has been successfully implemented.

devops user exists:

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "id devops" --become --vault-password-file .secrets/vault_pass.txt
```

The *devops user* and *devops group* have been created and the user is added to the *sudo group* - *Users & Groups* automation requirement satisfied.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "id devops" --become --vault-password-file .secrets/vault_pass.txt
ubuntu-prod | CHANGED | rc=0 >>
uid=1001(devops) gid=1001(devops) groups=1001(devops),27(sudo)
ubuntu-dev | CHANGED | rc=0 >>
uid=1001(devops) gid=1001(devops) groups=1001(devops),27(sudo)
```

become_user demo file exists:

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "ls -l /home/devops/become_user_demo.txt" --become --vault-password-file .se
crets/vault_pass.txt
```

The file's owner/group being *devops:devops* proves that the task was executed in the *become_user=devops* context (privilege demo successful).

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "ls -l /home/devops/become_user_demo.txt" --become --vault-password-file .se
crets/vault_pass.txt
ubuntu-prod | CHANGED | rc=0 >>
-rw-r--r-- 1 devops devops 47 Jan 28 19:35 /home/devops/become_user_demo.txt
ubuntu-dev | CHANGED | rc=0 >>
-rw-r--r-- 1 devops devops 47 Jan 28 19:35 /home/devops/become_user_demo.txt
```

Checking NTP sources:

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "chronyc sources -v | head -n 15" --become --vault-password-file .secrets/va
ult_pass.txt
```

chronyc sources -v output came in on both hosts and ^* is visible in the output

^* means Chrony has selected a best/current time source which means NTP sync is active and Chrony advanced scenario is successful.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "chronyc sources -v | head -n 15" --become --vault-password-file .secrets/va
ult_pass.txt
ubuntu-prod | CHANGED | rc=0 >>

  .-- Source mode  '^' = server, '=' = peer, '#' = local clock.
 / .- Source state '*' = current best, '+' = combined, '-' = not combined,
| /             'x' = may be in error, '~' = too variable, '?' = unusable.
||                                                 .- xxxx [ yyyy ] +/- zzzz
||      Reachability register (octal) -.           |  xxxx = adjusted offset,
||      Log2(Polling interval) --.      |          |  yyyy = measured offset,
||                                \     |          |  zzzz = estimated error.
||                                 |    |           \
MS Name/IP address         Stratum Poll Reach LastRx Last sample
===============================================================================
^- mail.rashnikov.name           2   9   377    59   +280us[ +280us] +/-   59ms
^* 51.250.35.68                  2  10   377   697  +1105us[ +860us] +/-   15ms
^+ 82.202.138.88                 4   7   377     5  -5130us[-5130us] +/-   31ms
^+ 83.237.235.30                 2   8   377   418  -3342us[-3342us] +/-   29ms
^+ time3.google.com              1  10   377   722  +1599us[+1354us] +/-   22ms
^+ time4.google.com              1  10   377   611  +2753us[+2753us] +/-   22ms
^+ time1.google.com              1  10   377   697  +2010us[+1765us] +/-   19ms
^+ time2.google.com              1  10   377   169  -1048us[-1048us] +/-   21ms
ubuntu-dev | CHANGED | rc=0 >>
```

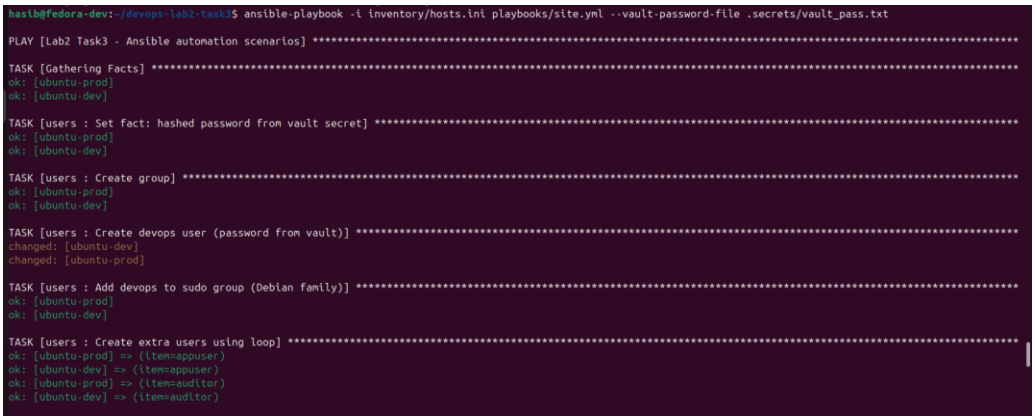
```
ubuntu-dev | CHANGED | rc=0 >>

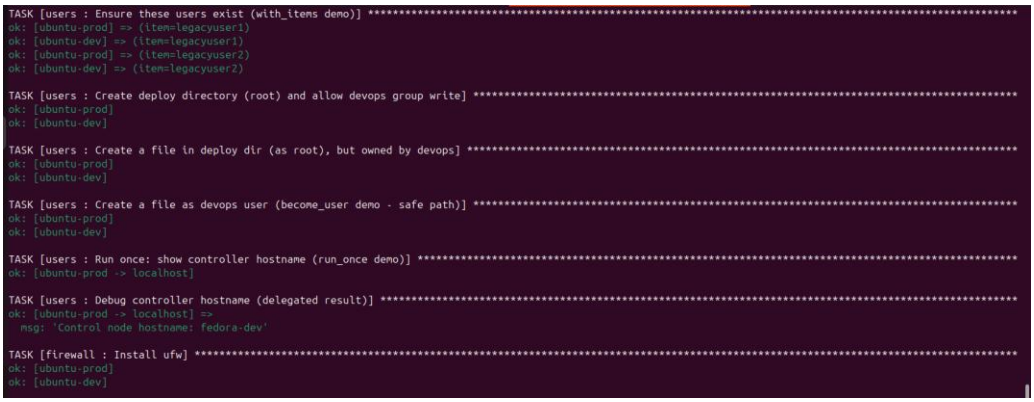
  .-- Source mode  '^' = server, '=' = peer, '#' = local clock.
 / .- Source state '*' = current best, '+' = combined, '-' = not combined,
| /             'x' = may be in error, '~' = too variable, '?' = unusable.
||                                                 .- xxxx [ yyyy ] +/- zzzz
||      Reachability register (octal) -.           |  xxxx = adjusted offset,
||      Log2(Polling interval) --.      |          |  yyyy = measured offset,
||                                \     |          |  zzzz = estimated error.
||                                 |    |           \
MS Name/IP address         Stratum Poll Reach LastRx Last sample
===============================================================================
^* 51.250.35.68                  2   8   377   464    +81us[ +602us] +/-   15ms
^+ 82.202.138.88                 4   8   377    64  -3709us[-3709us] +/-   29ms
^+ mail.rashnikov.name           2   8   377   176    -27us[  -27us] +/-   58ms
^+ 83.237.235.30                 2  10   377   119  -5234us[-5234us] +/-   31ms
^+ time1.google.com              1  10   377   713  +1199us[+1724us] +/-   19ms
^+ time3.google.com              1  10   377   642   +484us[+1009us] +/-   22ms
^+ time2.google.com              1  10   377   712  -1481us[ -955us] +/-   21ms
^+ time4.google.com              1  10   377   694  -1972us[-1447us] +/-   21ms
hasib@fedora-dev:~/devops-lab2-task3$
```

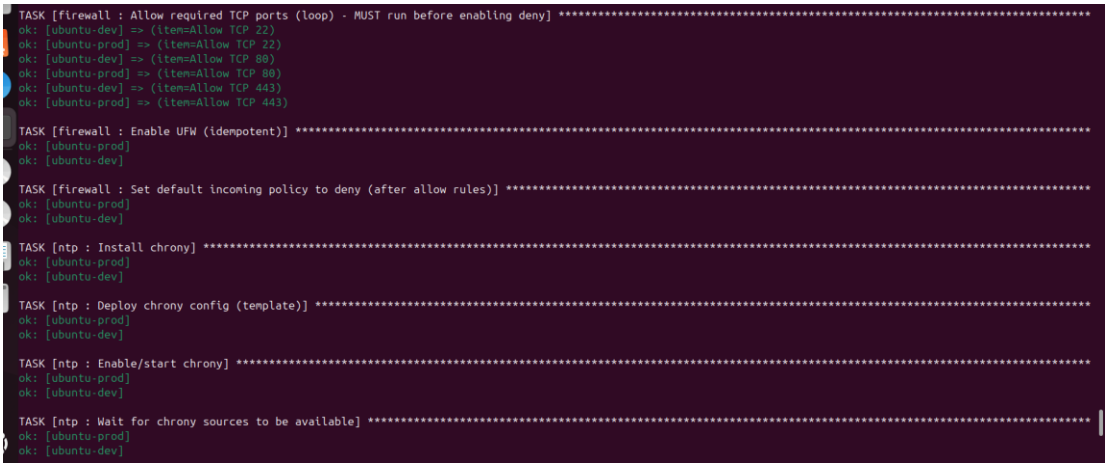
Full playbook run:

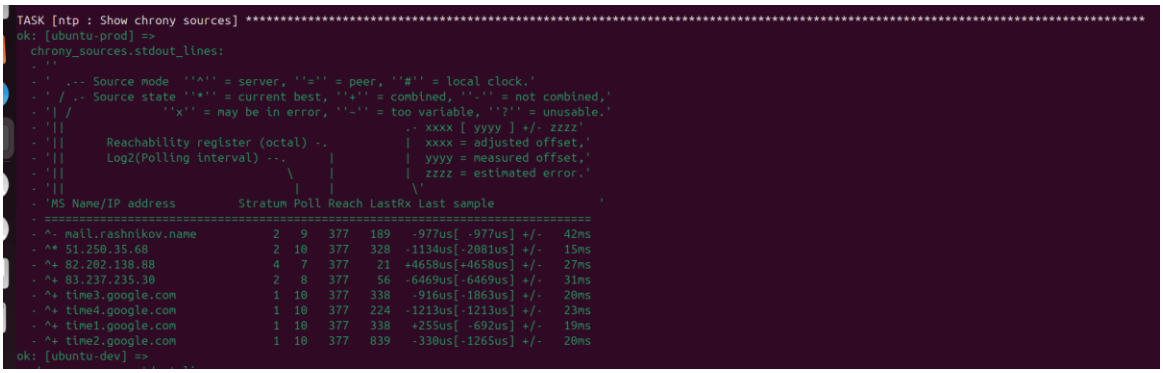
```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --vault-password-file .secrets/vault_pass.txt
```

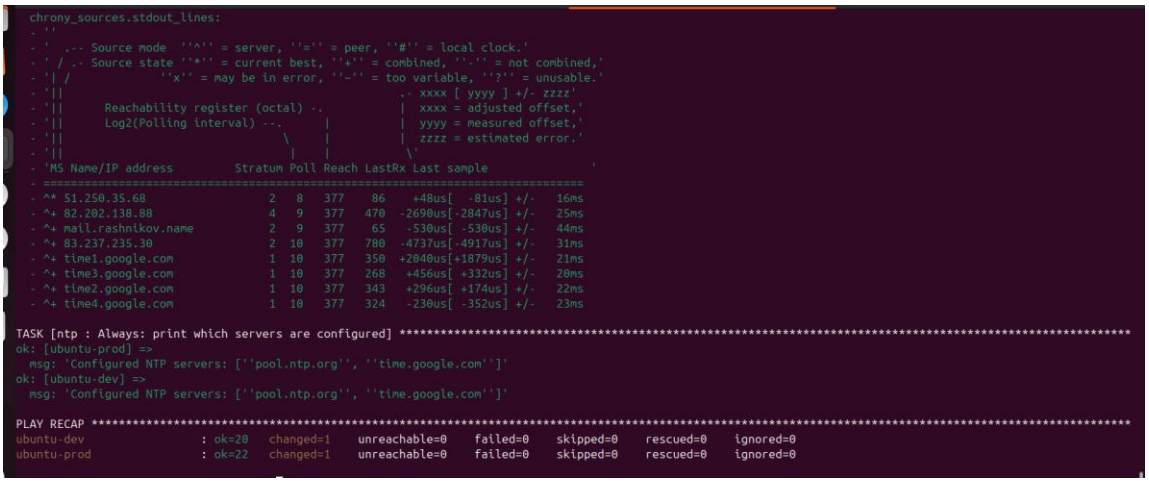
In this step, the entire *site.yml* playbook is run end-to-end again, verifying that all roles including users, firewall, and ntp are orchestrated and working properly. *PLAY RECAP* shows *unreachable=0* and *failed=0*, meaning that no hosts are unreachable, no tasks fail, and all roles, including Chrony verification, are executed successfully.





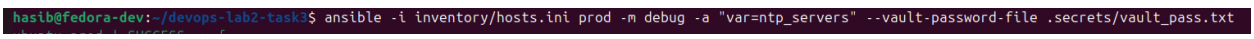




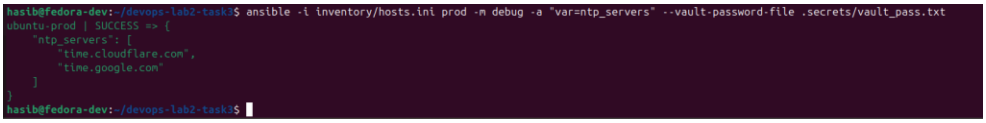


NTP override:

This verification inspects the *ntp_servers* variable directly for the prod group. The output shows *time.cloudflare.com* and *time.google.com*, which are the values defined in *group_vars/prod/main.yml.*



The output shows that the variable precedence of the prod group is effective, with *group_vars/prod* successfully overriding *group_vars/all.*



Git Repository

In this section, the Ansible automation project has been brought under version control in a Git repository and pushed to GitHub.

Initialize git (project root)

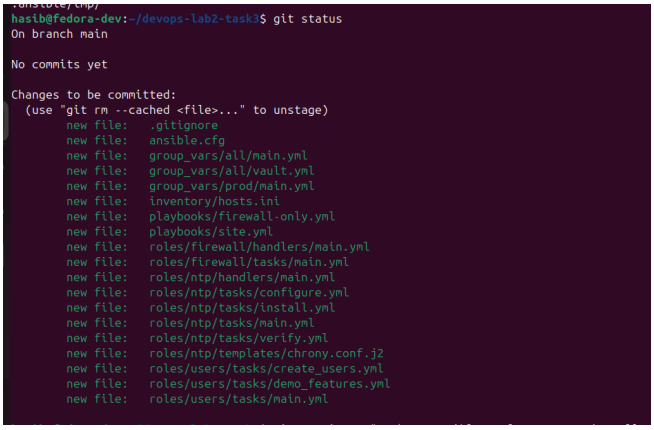
At this stage, the Git repository has been successfully initialized by running *git init* in the project root directory. The git status output shows that there are no commits yet and all the important project directories *ansible.cfg, group_vars/, inventory/, playbooks/, roles/* are in untracked state, which is normal because they have not been added to Git yet.



The *.secrets/ directory* also appears to be untracked.



This is excluded in the next step with *.gitignore* so that the Vault password or sensitive information is never committed to the repository.



Configuring Git identity:

```
hasib@fedora-dev:~/devops-lab2-task3$ git config --global user.name "Hasib Al Tahsin"
```

In this step, the user identity is configured one-time before the Git commit. Setting *git config --global user.name* and *git* config *--global user.email* ensures that the name and email are correctly recorded as the author in every future commit.

```
hasib@fedora-dev:~/devops-lab2-task3$ git config --global user.email "hasibjust09@gmail.com"
```

To check:

```
hasib@fedora-dev:~/devops-lab2-task3$ git config --global --list
```

Verifying with git config *--global --list* confirmed that the configuration was successfully applied, which was important for maintaining a clean and traceable commit history.

```
hasib@fedora-dev:~/devops-lab2-task3$ git config --global --list
user.name=Hasib Al Tahsin
user.email=hasibjust09@gmail.com
```

Branch name master - *main* :

```
hasib@fedora-dev:~/devops-lab2-task3$ git branch -m main
```

Main is used as the default branch.

.gitignore **verification:**

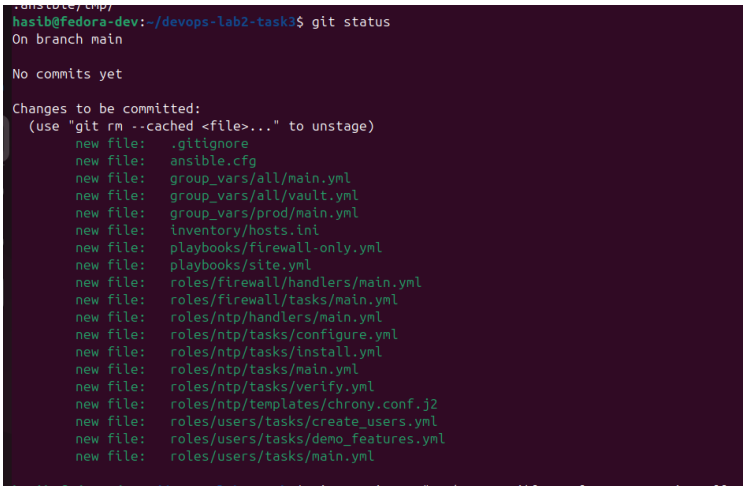
```
hasib@fedora-dev:~/devops-lab2-task3$ cat .gitignore
```

Here *.secrets/* is excluded to protect vault password, sensitive data and cache, temp, retry files will not go into the repo. This ensures Git hygiene.

```
hasib@fedora-dev:~/devops-lab2-task3$ cat .gitignore
.secrets/
*.retry
*.log
*.pyc
__pycache__/
.cache/
.ansible/tmp/
```

Re-checking Git status:

Ready to commit all required automation files for the project.

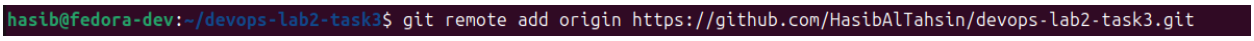


Create a new repository on GitHub:

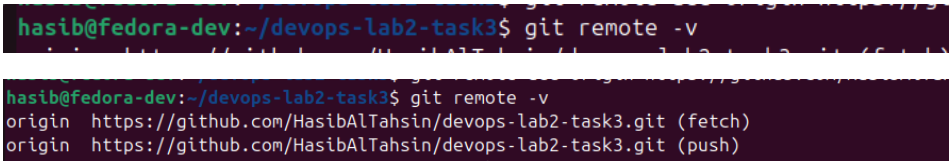
After going to the site *https://github.com/HasibAlTahsin* in the browser, a new repository was opened and named *devops-lab2-task3*. The local repository is the source of truth, so no pre-initialized file was kept on GitHub.

Connecting local repo - GitHub repo:

In this step, the local Git repository has been successfully connected to the GitHub remote. The GitHub repository URL has been added using the git remote add origin command using the following command.



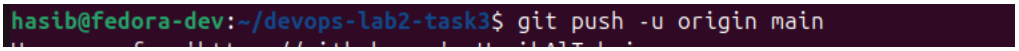
The following command verifies that the same remote is set for both fetch and push.



The local repo is now properly linked to GitHub.

Pushing the code:

In this step, the project has been successfully uploaded to GitHub by doing a final push from the local repository. Running the following command has created the main branch and started tracking it with main.



During the first push, GitHub requested authentication, using the GitHub Personal Access Token (PAT) as the password.

```
hasib@fedora-dev:~/devops-lab2-task3$ git push -u origin main
Username for 'https://github.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@github.com':
Enumerating objects: 36, done.
Counting objects: 100% (36/36), done.
Delta compression using up to 4 threads
Compressing objects: 100% (29/29), done.
Writing objects: 100% (36/36), 5.30 KiB | 63.00 KiB/s, done.
Total 36 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/HasibAlTahsin/devops-lab2-task3.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Github link: https://github.com/HasibAlTahsin/devops-lab2-task3

Conclusion for Task 3:

Task 3 *Control node (fedora-dev*) has been created following best-practice structured project layout (*inventory/playbooks/roles/group_vars/.secrets*), environment separation has been done with prod and dev groups in inventory and automation has been made completely non-interactive and secure using SSH key-based authentication + Vault-based secrets. The *site.yml* playbook has been designed as an entry point and three roles users, firewall and *ntp* have been orchestrated together, along with selective run using tags. In real execution, ping/become test, syntax-check, check-mode and full run were successful and pipeline stable was proven by showing *unreachable=0* and *failed=0*. Firewall policy (*deny incoming + only 22/80/443 allow),* Chrony NTP sync verification (^* in chronyc output) and prod-only *ntp_servers* override debug output have all been added to the report as end-to-end proof. As a bonus work, ansible-lint was installed, runtime/vault issues were troubleshooted, and a clean pass of 0 failures + 0 warnings (production profile) was ensured.

Bonus 1 - Ansible Collections (Galaxy)

In this section, an external Ansible Collection (community.general) is selected, installed, and verified using Ansible Galaxy.

Control node: fedora-dev

All Galaxy/collection-related work is done from the project root.

```
hasib@fedora-dev:~/devops-lab2-task3$ pwd
/home/hasib/devops-lab2-task3
```

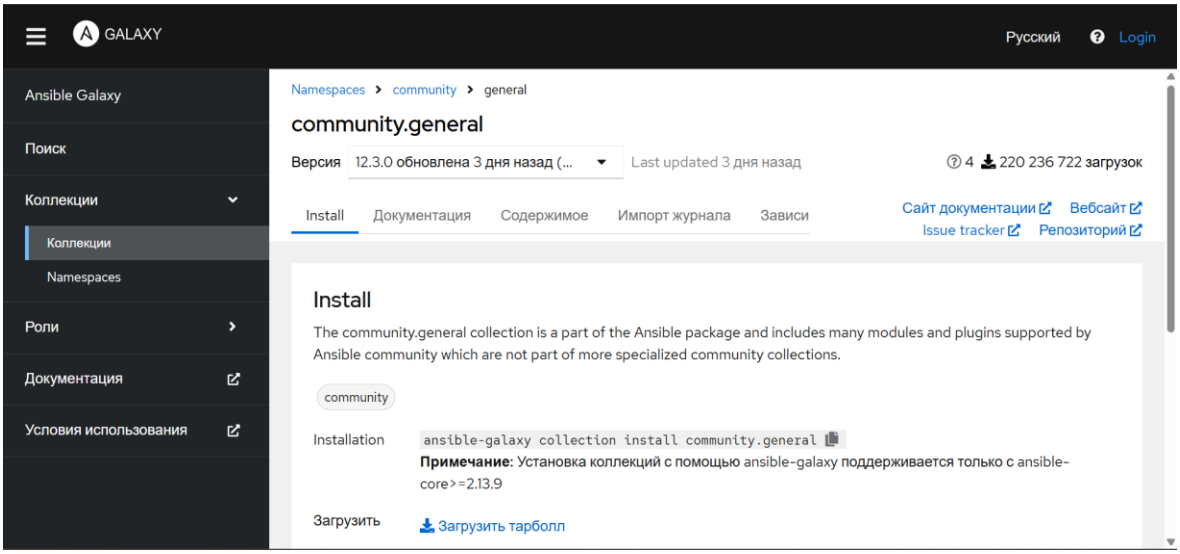
Galaxy UI follow (Collections):

Galaxy UI open:

Opened Ansible Galaxy UI from browser: *https://galaxy.ansible.com/ui/*

Opened Collection page and selected *community.general* from there.

https://galaxy.ansible.com/ui/repo/published/community/general/



Creating *requirements.yml* in the project:

It is recommended to use *requirements.yml* to install the collection.

Create directory:

```
hasib@fedora-dev:~/devops-lab2-task3$ mkdir -p collections
```

File creation:

```
hasib@fedora-dev:~/devops-lab2-task3$ nano collections/requirements.yml
```

collections/requirements.yml is used to make the project reproducible. This allows you to clone the repo and install all the required collections with a single command (ansible-galaxy collection *install -r collections/requirements.yml*).

```
 GNU nano 7.2                                  collections/requirements.yml
---
collections:
  - name: community.general
    version: "*"
```

Install collection:

Install from *requirements.yml:*

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-galaxy collection install -r collections/requirements.yml
Starting galaxy collection install process
Nothing to do. All requested collections are already installed. If you want to reinstall them, consider using `--force`.
```

Direct install:

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-galaxy collection install community.general
Starting galaxy collection install process
Nothing to do. All requested collections are already installed. If you want to reinstall them, consider using `--force`.
```

This output shows that the collection was already installed. This means the environment is ready and consistent.

Installed collection verify:

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-galaxy collection list | grep -E '^community\.general'
community.general                 8.3.0
```

The exact collection name and version are visible in the output

Project-local collection installing:

Create local collections directory:

```
hasib@fedora-dev:~/devops-lab2-task3$ mkdir -p collections/ansible_collections
```

In this step, the collection is installed to the local project path using *requirements.yml*, so that there is no dependency on the *system/global* path. The output shows that *community.general:8.6.0* was successfully installed to the *collections/ansible_collections/... directory.*

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-galaxy collection install -r collections/requirements.yml -p collections/ansible_collections
```

The warning that came is also expected. Because by default Ansible does not know this custom path yet, so it says that the path is not part of the configured collections paths.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-galaxy collection install -r collections/requirements.yml -p collections/ansible_collections
Starting galaxy collection install process
[WARNING]: The specified collections path '/home/hasib/devops-lab2-task3/collections/ansible_collections' is not part of the configured Ansible collections paths
'/home/hasib/.ansible/collections:/usr/share/ansible/collections'. The installed collection will not be picked up in an Ansible run, unless within a playbook-
adjacent collections directory.
Process install dependency map
Starting collection install process
Downloading https://galaxy.ansible.com/api/v3/plugin/ansible/content/published/collections/artifacts/community-general-8.6.0.tar.gz to /home/hasib/.ansible/tmp/ans
ible-local-528537q7c1ssm/tmpj9dlyzse/community-general-8.6.0-yr6h8jiv
Installing 'community.general:8.6.0' to '/home/hasib/devops-lab2-task3/collections/ansible_collections/community/general'
community.general:8.6.0 was installed successfully
```

Adding collections paths to ansible.cfg:

Added *collections_paths* to the [defaults] section of *ansible.cfg* so that Ansible knows where to load collections from. The order of the paths here is very arbitrary.

```
hasib@fedora-dev:~/devops-lab2-task3$ nano ansible.cfg
```

First, *./collections/ansible_collections* is placed so that project-local collections get the highest priority. Then *~/.ansible/collections* is placed for user-level collections, and finally */usr/share/ansible/collections* is placed as a system-wide fallback.

```
collections_paths = ./collections/ansible_collections:~/.ansible/collections:/usr/share/ansible/collections

[privilege_escalation]
```

This order ensures that the project-specific collection/version is picked first, without conflicting with the system or user environment.

Verifying collections path is active:

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-config dump | grep -i collections_paths
```

The *collections_path* is in the future and is currently valid. The output shows that Ansible successfully picked the project-local path.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-config dump | grep -i collections_paths
[DEPRECATION WARNING]: [defaults]collections_paths option, does not fit var
naming standard, use the singular form collections_path instead. This feature
will be removed from ansible-core in version 2.19. Deprecation warnings can be
disabled by setting deprecation_warnings=False in ansible.cfg.
COLLECTIONS_PATHS(/home/hasib/devops-lab2-task3/ansible.cfg) = ['/home/hasib/devops-lab2-task3/collections/ansible_collections', '/home/hasib/.ansible/collections'
, '/usr/share/ansible/collections']
```

Collection module usage:

Using community.general.ufw in UFW role:

```
hasib@fedora-dev:~/devops-lab2-task3$ grep -RIn "community\.general\.ufw" roles/firewall/tasks/main.yml
```

The *community.general.ufw* module is actively used in firewall automation and both collection install + consume have been done.

```
hasib@fedora-dev:~/devops-lab2-task3$ grep -RIn "community\.general\.ufw" roles/firewall/tasks/main.yml
8:  community.general.ufw:
17:  community.general.ufw:
21:  community.general.ufw:
```

ansible-doc:

Ansible-doc is used to verify that the Collection module is accessible to the Ansible runtime.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-doc community.general.ufw | head -n 40
```

Module path is visible in project-local directory and Ansible successfully loaded documentation. Collection usable and properly registered.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-doc community.general.ufw | head -n 40
[DEPRECATION WARNING]: [defaults]collections_paths option, does not fit var
naming standard, use the singular form collections_path instead. This feature
will be removed from ansible-core in version 2.19. Deprecation warnings can be
disabled by setting deprecation_warnings=False in ansible.cfg.
> COMMUNITY.GENERAL.UFW    (/home/hasib/devops-lab2-task3/collections/ansible_collections/community/general/plugins/modules/ufw.py)

       Manage firewall with UFW.

OPTIONS (= is mandatory):

- comment
       Add a comment to the rule. Requires UFW version >=0.35.
       default: null
       type: str

- default
       Change the default policy for incoming or outgoing traffic.
       aliases: [policy]
       choices: [allow, deny, reject]
       default: null
       type: str

- delete
       Delete rule.
       If `delete=true' and a value is provided for `insert', then
       `insert' is ignored.
       default: false
       type: bool
```

```
- direction
       Select direction for a rule or default policy command.
       Mutually exclusive with `interface_in' and `interface_out'.
       choices: [in, incoming, out, outgoing, routed]
       default: null
       type: str

- from_ip
       Source IP address.
       aliases: [from, src]
       default: any
       type: str

- from_port
       Source port.
```

Firewall tag run:

Because *community.general.ufw* module is used in *roles/firewall/tasks/main.yml* file. So, firewall tag run means Galaxy collection module real execution proof.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --tags firewall --vault-password-file .secrets/vault_pass.txt
```

This output summary shows that the firewall automation end-to-end worked successfully. The two host facts were gathered correctly, UFW was installed, the required *port 22/80/443* was allowed, and finally the firewall was enabled with the default incoming policy deny set.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible-playbook -i inventory/hosts.ini playbooks/site.yml --tags firewall --vault-password-file .secrets/vault_pass.txt
[DEPRECATION WARNING]: [defaults]collections_paths option, does not fit var naming standard, use the singular form collections_path instead. This feature will be
removed from ansible-core in version 2.19. Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.

PLAY [Lab2 Task3 - Ansible automation scenarios] ********************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [firewall : Install ufw] **************************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [firewall : Allow required TCP ports (loop) - MUST run before enabling deny] **********************************************************************************
ok: [ubuntu-prod] => (item=Allow TCP 22)
ok: [ubuntu-dev] => (item=Allow TCP 22)
ok: [ubuntu-prod] => (item=Allow TCP 80)
ok: [ubuntu-dev] => (item=Allow TCP 80)
ok: [ubuntu-prod] => (item=Allow TCP 443)
ok: [ubuntu-dev] => (item=Allow TCP 443)
```

```
TASK [firewall : Enable UFW (idempotent)] **************************************************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

TASK [firewall : Set default incoming policy to deny (after allow rules)] ******************************************************************************************
ok: [ubuntu-prod]
ok: [ubuntu-dev]

PLAY RECAP *********************************************************************************************************************************************************
ubuntu-dev                 : ok=5    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu-prod                : ok=5    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

According to the output, two host facts have been gathered, UFW is installed, *22/80/443 port* is allowed, and the firewall is enabled with default incoming deny set. The changed=0 shown in PLAY RECAP means that the firewall was already in the expected state-rerunning the playbook did not make any unnecessary changes.

Verifying UFW status:

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "ufw status verbose" --become --vault-password-file .secrets/vault_pass.txt
```

This step verifies the post-run firewall state to ensure that the automation has actually been applied correctly. The *ufw status verbose* output for both hosts shows firewall active, default incoming policy deny, and only *22/80/443 (SSH/HTTP/HTTPS*) allowed.

```
hasib@fedora-dev:~/devops-lab2-task3$ ansible -i inventory/hosts.ini all -m command -a "ufw status verbose" --become --vault-password-file .secrets/vault_pass.txt
[DEPRECATION WARNING]: [defaults]collections_paths option, does not fit var naming standard, use the singular form collections_path instead. This feature will be
removed from ansible-core in version 2.19. Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.
ubuntu-prod | CHANGED | rc=0 >>
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)
ubuntu-dev | CHANGED | rc=0 >>
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip
```

```
To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)
ubuntu-dev | CHANGED | rc=0 >>
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)
```

Lint:

This bonus lint proof shows that the project passed ansible-lint cleanly despite having Galaxy collection, Vault, custom paths. The lint was run with *ANSIBLE_CONFIG* and *ANSIBLE_VAULT_PASSWORD_FILE* explicitly set, so the tool used the correct project config and Vault secret. This lint proof shows that the project passed ansible-lint cleanly despite having Galaxy collection, Vault, custom paths. The lint was run with *ANSIBLE_CONFIG* and *ANSIBLE_VAULT_PASSWORD_FILE* explicitly set, so the tool used the correct project config and Vault secret.

```
hasib@fedora-dev:~/devops-lab2-task3$ ANSIBLE_CONFIG="$PWD/ansible.cfg" \
ANSIBLE_VAULT_PASSWORD_FILE="$PWD/.secrets/vault_pass.txt" \
ansible-lint --project-dir "$PWD" playbooks/site.yml
```

0 failures / 0 warnings in the output means that all lint rules are satisfied, and passing the 'production' profile means that the stricter quality ruleset is also met. The deprecation or offline warnings that are received are not execution-blocking, but lint validation has been completed successfully.

```
hasib@fedora-dev:~/devops-lab2-task3$ ANSIBLE_CONFIG="$PWD/ansible.cfg" \
ANSIBLE_VAULT_PASSWORD_FILE="$PWD/.secrets/vault_pass.txt" \
ansible-lint --project-dir "$PWD" playbooks/site.yml
[DEPRECATION WARNING]: [defaults]collections_paths option, does not fit var
naming standard, use the singular form collections_path instead. This feature
will be removed from ansible-core in version 2.19. Deprecation warnings can be
disabled by setting deprecation_warnings=False in ansible.cfg.
[DEPRECATION WARNING]: [defaults]collections_paths option, does not fit var
naming standard, use the singular form collections_path instead. This feature
will be removed from ansible-core in version 2.19. Deprecation warnings can be
disabled by setting deprecation_warnings=False in ansible.cfg.
WARNING  Skipped installing collection dependencies due to running in offline mode.

Passed: 0 failure(s), 0 warning(s) on 11 files. Last profile that met the validation criteria was 'production'.
```

Git status checking:

```
hasib@fedora-dev:~/devops-lab2-task3$ git status
```

ansible.cfg was modified earlier due to adding the collections path. The collections/ directory is new and has not been committed yet, so staging was required.

```
hasib@fedora-dev:~/devops-lab2-task3$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ansible.cfg

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        collections/

no changes added to commit (use "git add" and/or "git commit -a")
```

Staging files:

```
hasib@fedora-dev:~/devops-lab2-task3$ git add collections/requirements.yml ansible.cfg roles/firewall/tasks/main.yml
```

This step involves selective staging, so that only the necessary config files go to Git. The dependencies are declared in collections/requirements.yml, the collections path and Vault config are in ansible.cfg, and the Galaxy module usage is proven in roles/firewall/tasks/main.yml, so all three are staged. collections/ansible_collections/ was not added intentionally, because it is a downloaded dependency. It was regenerated with requirements.yml instead of putting it in the repo.

Stage verifying:

This git status output is proof of clean staging.

```
hasib@fedora-dev:~/devops-lab2-task3$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   ansible.cfg
        new file:   collections/requirements.yml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        collections/ansible_collections/
```

.cfg was the project-level config update correctly staged and *collections/requirements.yml* was the Galaxy dependency declaration staged. *collections/ansible_collections/* was intentionally left untracked because it was a downloaded dependency.

Commit:

```
hasib@fedora-dev:~/devops-lab2-task3$ git commit -m "Bonus: Add Ansible Galaxy collection requirements and use community.general.ufw"
```

The Commit message at this stage clearly states that this is Bonus work, where Ansible Galaxy collection requirements have been added and evidence of the use of the *community.general.ufw* firewall module has been included. Since only the necessary files have been committed, the history is concise and readable.

```
hasib@fedora-dev:~/devops-lab2-task3$ git commit -m "Bonus: Add Ansible Galaxy collection requirements and use community.general.ufw"
[main e581db2] Bonus: Add Ansible Galaxy collection requirements and use community.general.ufw
 2 files changed, 6 insertions(+)
 create mode 100644 collections/requirements.yml
```

Push:

Push before safety checking (rebase):

The following command is run after the final push so that if there are new commits on the remote, then your work is cleanly applied to them.

```
hasib@fedora-dev:~/devops-lab2-task3$ git pull --rebase origin main
```

This prevents accidental merge commits and keeps Git history linear and tidy.

```
hasib@fedora-dev:~/devops-lab2-task3$ git pull --rebase origin main
remote: Enumerating objects: 16, done.
remote: Counting objects: 100% (16/16), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 10 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (10/10), 2.20 KiB | 54.00 KiB/s, done.
From https://github.com/HasibAlTahsin/devops-lab2-task3
 * branch            main       -> FETCH_HEAD
   70bd4fa..ae80a59  main       -> origin/main
Successfully rebased and updated refs/heads/main.
```

Final push to GitHub:

```
hasib@fedora-dev:~/devops-lab2-task3$ git push origin main
```

GitHub Personal Access Token (PAT) is used for authentication.

```
hasib@fedora-dev:~/devops-lab2-task3$ git push origin main
Username for 'https://github.com': HasibAlTahsin
Password for 'https://HasibAlTahsin@github.com':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 593 bytes | 65.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/HasibAlTahsin/devops-lab2-task3.git
   ae80a59..d11bca8  main -> main
```

Verify after push:

The output of the following command clearly shows that the Bonus commit has been successfully added to the main branch, all important commits from the previous Task-3 are intact, and local main is fully synced with main.

```
hasib@fedora-dev:~/devops-lab2-task3$ git log --oneline -5
```

The output below clearly shows that the Bonus commit has been successfully added to the main branch, all important commits from the previous Task-3 are intact, and local main is fully synced with main.

```
hasib@fedora-dev:~/devops-lab2-task3$ git log --oneline -5
d11bca8 (HEAD -> main, origin/main) Bonus: Add Ansible Galaxy collection requirements and use community.general.ufw
ae80a59 Update NTP servers configuration
495841e Update main.yml
70bd4fa Task 3: Ansible Vault, Users, Firewall, NTP, Lint-compliant roles
```

Conclusion for the Bonus Task 1:

In this task, the end-to-end verification has been completed by successfully selecting, installing, configuring and consuming the external collection *community.general* from Ansible Galaxy. To keep the project reproducible, *collections/requirements.yml* has been created, so that the dependency can be installed in a single command after cloning the repo. The collection has been installed from the requirements and, as a clean approach, also installed in the project-local path (*collections/ansible_collections*) and *collections_paths* have been added to *ansible.cfg*, so that the project-specific version gets the highest priority. Then, the policy has been applied in reality using the *community.general.ufw* module in the firewall role and the module availability proof has been shown with ansible-doc *community.general.ufw*. By running *--tags* firewall and verifying *ufw status verbose*, it has been proven that the collection module has been real executed and the desired firewall state (*deny incoming + allow 22/80/443*) is maintained. Finally, the required files (*requirements.yml + ansible.cfg + firewall task*) have been cleanly committed/pushed to Git and the dependency directory (*collections/ansible_collections/)* has been intentionally kept out of version control, keeping the repo lightweight and clean

Bonus 2 - Ansible AWX

Kubernetes (K3s) Installation:

Kubernetes is required to run AWX Operator. K3s is a lightweight Kubernetes distribution that allows you to quickly set up a cluster on a single VM.

```
hasib@fedora-dev:~/devops-lab2-task3$ curl -sfL https://get.k3s.io | sh -
[sudo] password for hasib:
[INFO]  Finding release for channel stable
[INFO]  Using v1.34.3+k3s1 as release
[INFO]  Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.34.3+k3s1/sha256sum-amd64.txt
[INFO]  Skipping binary downloaded, installed k3s matches hash
[INFO]  Skipping installation of SELinux RPM
[INFO]  Skipping /usr/local/bin/kubectl symlink to k3s, command exists in PATH at /usr/bin/kubectl
[INFO]  Skipping /usr/local/bin/crictl symlink to k3s, already exists
[INFO]  Skipping /usr/local/bin/ctr symlink to k3s, command exists in PATH at /usr/bin/ctr
[INFO]  Creating killall script /usr/local/bin/k3s-killall.sh
[INFO]  Creating uninstall script /usr/local/bin/k3s-uninstall.sh
[INFO]  env: Creating environment file /etc/systemd/system/k3s.service.env
[INFO]  systemd: Creating service file /etc/systemd/system/k3s.service
[INFO]  systemd: Enabling k3s unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s.service → /etc/systemd/system/k3s.service.
[INFO]  No change detected so skipping service start
```

The script is safely fetched using *curl -sfL* and the script is run using the shell using | *sh*.

Cluster verifying:

```
hasib@fedora-dev:~/devops-lab2-task3$ kubectl get nodes
```

The output shows that Node Ready and cluster is OK and control-plane was this node. Kubernetes version was *v1.34.3+k3s1* K3s cluster was running successfully.

```
hasib@fedora-dev:~/devops-lab2-task3$ kubectl get nodes
NAME         STATUS   ROLES           AGE   VERSION
fedora-dev   Ready    control-plane   24h   v1.34.3+k3s1
```

Required Tools Installation:

Installing Git:

It took Git to clone AWX Operator from GitHub. *-y* is used to complete the install without prompting.

```
hasib@fedora-dev:~/devops-lab2-task3$ sudo apt install -y git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
0 upgraded, 0 newly installed, 0 to remove and 155 not upgraded.
```

kubectl installing:

This step ensures that kubectl is present on the system. If it is not present, it is installed using the following command.

```
hasib@fedora-dev:~/devops-lab2-task3$ sudo snap install kubectl --classic
```

The output shows that *kubectl 1.34.3* has been successfully installed from Canonical's snap package. This makes the CLI tool required to interact with the Kubernetes cluster ready.

```
hasib@fedora-dev:~/devops-lab2-task3$ sudo snap install kubectl --classic
kubectl 1.34.3 from Canonical✓ installed
WARNING: There are 2 new warnings. See 'snap warnings'.
```

Verifying:

Running *kubectl version --client* verified that *kubectl* is properly installed and usable.

```
hasib@fedora-dev:~/devops-lab2-task3$ sudo snap install kubectl --classic
```

The output showed *Client Version v1.30.14* and *Kustomize v5.0.4.*

```
hasib@fedora-dev:~/devops-lab2-task3$ kubectl version --client
Client Version: v1.30.14
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
```

Cloning AWX GitHub Repository:

AWX source + operator manifests + examples are needed. The official repo has been cloned.

```
hasib@fedora-dev:~/devops-lab2-task3$ git clone https://github.com/ansible/awx.git
cd awx
Cloning into 'awx'...
remote: Enumerating objects: 329060, done.
remote: Counting objects: 100% (139/139), done.
remote: Compressing objects: 100% (70/70), done.
remote: Total 329060 (delta 95), reused 69 (delta 69), pack-reused 328921 (from 2)
Receiving objects: 100% (329060/329060), 345.38 MiB | 1.81 MiB/s, done.
Resolving deltas: 100% (252344/252344), done.
Updating files: 100% (1760/1760), done.
```

This repository contains:

- AWX source
- Operator manifests
- Deployment examples

Creating AWX Namespace:

The *awx* namespace has been created to keep resources (operator, *awx* instance, secrets, services) separate in Kubernetes.

Verifying:

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl get ns | grep awx
awx               Active   24h
```

The output shows that the Namespace was ready.

Verifying AWX CRDs:

When the AWX Operator is installed, some Custom Resource Definitions (CRDs) are registered in Kubernetes, such as *awxs.awx.ansible.com*. The presence of these means that the Operator/CRD layer is present.

```
hasib@fedora-dev:~/devops-lab2-task3/awx/awx$ kubectl get crd | grep awx
```

In this step, the CRD layer of AWX Operator is verified. Running the above command shows CRDs like *awxs.awx.ansible.com*, *awxbackups*, *awxrestores* etc., which proves that AWX's Custom Resource Definitions have been successfully registered in the Kubernetes cluster. This means that the Operator/CRD infrastructure is present.

```
hasib@fedora-dev:~/devops-lab2-task3/awx/awx$ kubectl get crd | grep awx
awxbackups.awx.ansible.com                2026-01-30T01:59:43Z
awxmeshingresses.awx.ansible.com          2026-01-30T01:59:44Z
awxrestores.awx.ansible.com               2026-01-30T01:59:44Z
awxs.awx.ansible.com                      2026-01-30T01:59:44Z
```

Creating Admin Password Secret:

The admin password that would have been created when the AWX instance was created was already stored as a secret. The reason for using Kubernetes secrets was to avoid hardcoding the plaintext password into the *repo* or *yaml* and to provide cluster-level secured storage (RBAC + namespace scope).

```
hasib@fedora-dev:~/devops-lab2-task3/awx/awx$ kubectl -n awx create secret generic awx-admin-password \
  --from-literal=password='<STRONG_ADMIN_PASSWORD>'
error: failed to create secret secrets "awx-admin-password" already exists
```

awx-admin-password secret was already created. The create command cannot override an existing object, so it says "*already exists*".

Verifying:

```
hasib@fedora-dev:~/devops-lab2-task3/awx/awx$ kubectl -n awx get secret awx-admin-password
```

This verification indicates that the AWX admin password secret was successfully generated. Getting an Opaque type secret in the above command means that the AWX Operator has generated the admin credential and stored it in Kubernetes.

```
hasib@fedora-dev:~/devops-lab2-task3/awx/awx$ kubectl -n awx get secret awx-admin-password
NAME                TYPE     DATA   AGE
awx-admin-password   Opaque   1      25h
```

Creating AWX Deployment Manifest:

After installing AWX Operator, AWX Custom Resource (CR) had to be applied to deploy AWX. Applying this CR allowed Operator to automatically create all components of AWX (web, task, postgres, migration, services).

File created manually:

In this step, AWX Deployment Manifest is created in fedora-dev VM, from specific working directory *~/devops-lab2-task3/awx/awx.* Here, an AWX custom resource is defined in *awx-deploy.yml* file, which AWX Operator automatically starts AWX deploy.

```
hasib@fedora-dev:~/devops-lab2-task3/awx/awx$ nano awx-deploy.yml
```

apiVersion: *awx.ansible.com/v1beta1* is used because the Operator registered this CRD, and kind: AWX indicates that this is an AWX instance. *metadata.name: awx* and *namespace: awx* mean that all AWX-related pods, services, and secrets are created in that namespace.

Providing *spec.service_type:* nodeport allows for easy access to the AWX UI in the lab environment. *admin_user:* admin is used to set the admin username, and *admin_password_secret: awx-admin-password* is used to take the password from the Kubernetes secret, eliminating the need to hardcode the plaintext password in the manifest. This configuration ensured a secure, operator-driven, and lab-friendly AWX deployment.

```
 GNU nano 7.2                                                awx-deploy.yml *
apiVersion: awx.ansible.com/v1beta1
kind: AWX
metadata:
  name: awx
  namespace: awx
spec:
  service_type: nodeport
  admin_user: admin
  admin_password_secret: awx-admin-password
```

Applying manifest (AWX CR configure):

```
hasib@fedora-dev:~/devops-lab2-task3/awx/awx$ kubectl apply -f awx-deploy.yml
awx.awx.ansible.com/awx configured
```

configured means the resource has been updated or the same config has been re-applied. The operator therefore created/maintained AWX components by running the reconcile loop.

Verifying AWX CR exists:

```
hasib@fedora-dev:~/devops-lab2-task3/awx/awx$ kubectl -n awx get awx
NAME   AGE
awx    25h
```

The output shows that AWX custom resource is present in the cluster.

AWX Component Deployment:

After applying the AWX CR, the operator automatically created:

- awx-web
- awx-task
- awx-postgres
- awx-migration

Pods verification:

The above command is used to run a live watch to see the actual status of the AWX deployment in real-time. According to your output, at least one 3/3 of the *awx-web pod* is running. This is the strongest evidence that the web/UI component of AWX has started successfully and all containers are up.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get pods -w
```

awx-postgres-15-0 1/1 Running indicates that the database is running properly, and *awx-migration-24.6.1-...* Completed means that the migration job has completed successfully, so that the database schema/setup is complete. *Init:0/2, Init:1/2*, Pending, *ContainerStatusUnknown* in *awx-task* pods can usually be a cluster resource/runtime side issue. However, *awx-web Running + postgres Running + migration* Completed together means that the deployment has progressed significantly and the core components have been effectively deployed.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get pods -w
NAME                                               READY   STATUS                   RESTARTS      AGE
awx-migration-24.6.1-kpn2j                         0/1     Completed                0             11h
awx-operator-controller-manager-5974745896-bm5bw   2/2     Running                  0             12m
awx-operator-controller-manager-5974745896-d87wb   0/2     ContainerStatusUnknown   2             45m
awx-operator-controller-manager-5974745896-hv8k5   0/2     ContainerStatusUnknown   4 (64m ago)   69m
awx-postgres-15-0                                  1/1     Running                  0             11m
awx-task-764b55647d-6zxxn                          0/4     Init:ContainerStatusUnknown   1        65m
awx-task-764b55647d-pbrc9                          0/4     Init:0/2                 0             13m
awx-task-764b55647d-wjg99                          0/4     Init:ContainerStatusUnknown   1        45m
awx-task-8b49bd7f9-4lkrw                           0/4     Init:0/2                 0             13m
awx-task-8b49bd7f9-kjvtp                           0/4     Init:ContainerStatusUnknown   1        45m
awx-task-8b49bd7f9-ll68p                           0/4     Init:ContainerStatusUnknown   1        71m
awx-web-5db66ff9df-8twmq                           0/3     ContainerStatusUnknown   3             45m
awx-web-5db66ff9df-hdz25                           0/3     ContainerStatusUnknown   3 (60m ago)   67m
awx-web-5db66ff9df-zp5z2                           0/3     ContainerCreating        0             12m
awx-task-8b49bd7f9-4lkrw                           0/4     Init:0/2                 0             13m
awx-task-764b55647d-wjg99                          0/4     Init:ContainerStatusUnknown   1        45m
awx-migration-24.6.1-kpn2j                         0/1     Completed                0             11h
awx-task-8b49bd7f9-kjvtp                           0/4     Init:ContainerStatusUnknown   1        46m
awx-operator-controller-manager-5974745896-d87wb   0/2     ContainerStatusUnknown   2             45m
awx-task-764b55647d-pbrc9                          0/4     Init:0/2                 0             13m
awx-web-5db66ff9df-zp5z2                           1/3     ContainerCreating        0             12m
awx-task-8b49bd7f9-ll68p                           0/4     Init:ContainerStatusUnknown   1        71m
awx-task-764b55647d-6zxxn                          0/4     Init:ContainerStatusUnknown   1        66m
awx-operator-controller-manager-5974745896-hv8k5   0/2     ContainerStatusUnknown   4 (64m ago)   69m
awx-web-5db66ff9df-hdz25                           0/3     ContainerStatusUnknown   3 (61m ago)   68m
awx-web-5db66ff9df-8twmq                           0/3     ContainerStatusUnknown   3             45m
awx-operator-controller-manager-5974745896-bm5bw   1/2     Unknown                  0             13m
awx-operator-controller-manager-5974745896-bm5bw   1/2     Running                  1 (38s ago)   13m
awx-operator-controller-manager-5974745896-bm5bw   2/2     Running                  1 (40s ago)   13m
awx-task-764b55647d-pbrc9                          0/4     Init:0/2                 0             19m
```

```
awx-task-8b49bd7f9-s62h7                           0/4     Pending                  0             2s
awx-task-8b49bd7f9-kjvtp                           0/4     Init:ContainerStatusUnknown   1        65m
awx-task-764b55647d-8tvf7                          0/4     Init:0/2                 0             5s
awx-task-8b49bd7f9-ll68p                           0/4     Init:ContainerStatusUnknown   1        91m
awx-task-8b49bd7f9-ll68p                           0/4     Init:ContainerStatusUnknown   1        91m
awx-task-8b49bd7f9-s62h7                           0/4     Init:0/2                 0             14s
awx-operator-controller-manager-5974745896-bm5bw   1/2     Running                  2 (24s ago)   33m
awx-operator-controller-manager-5974745896-bm5bw   2/2     Running                  2 (29s ago)   33m
awx-web-5db66ff9df-8twmq                           0/3     ContainerStatusUnknown   3             66m
awx-task-764b55647d-8tvf7                          0/4     Init:0/2                 0             50s
awx-web-5db66ff9df-8twmq                           0/3     ContainerStatusUnknown   3             66m
awx-web-5db66ff9df-hdz25                           0/3     ContainerStatusUnknown   3 (81m ago)   88m
awx-web-5db66ff9df-hdz25                           0/3     ContainerStatusUnknown   3 (81m ago)   88m
awx-web-5db66ff9df-zp5z2                           3/3     Terminating              0             33m
awx-task-8b49bd7f9-s62h7                           0/4     Init:0/2                 0             52s
awx-web-5db66ff9df-zp5z2                           3/3     Terminating              0             33m
awx-web-5db66ff9df-jb926                           0/3     Pending                  0             1s
awx-web-5db66ff9df-jb926                           0/3     Pending                  0             4s
awx-web-5db66ff9df-jb926                           0/3     ContainerCreating        0             12s
awx-operator-controller-manager-7887dd74b-gfsnj    0/2     Pending                  0             5s
awx-operator-controller-manager-7887dd74b-gfsnj    0/2     Pending                  0             12s
awx-operator-controller-manager-7887dd74b-gfsnj    0/2     ContainerCreating        0             18s
awx-task-764b55647d-8tvf7                          0/4     Init:1/2                 0             2m5s
awx-operator-controller-manager-5974745896-bm5bw   1/2     Error                    2 (2m2s ago)   35m
awx-operator-controller-manager-5974745896-bm5bw   1/2     Running                  3 (61s ago)    35m
awx-task-8b49bd7f9-s62h7                           0/4     Init:1/2                 0             2m41s
awx-operator-controller-manager-5974745896-bm5bw   2/2     Running                  3 (70s ago)    35m
awx-operator-controller-manager-7887dd74b-gfsnj    2/2     Running                  0             74s
awx-operator-controller-manager-5974745896-bm5bw   2/2     Terminating              3 (83s ago)    35m
awx-web-5db66ff9df-jb926                           3/3     Running                  0             2m7s
awx-operator-controller-manager-5974745896-bm5bw   2/2     Terminating              3 (89s ago)    35m
```

Service Exposure (NodePort):

AWX Operator exposes *awx-service* as *NodePort* so that AWX Web UI can be accessed from browser.

In this step, *awx-service* details are verified. *TYPE: NodePort* in Output shows that the service is exposed on a high port of node. *PORT(S) 80:31010/TCP* means that AWX web service is running on *port 80* inside the cluster, and *port 31010* has been opened on node for access from outside.

```
hasib@fedora-dev:~/devops-lab2-task3/awx/awx$ kubectl -n awx get svc awx-service -o wide
```

The SELECTOR mentions *awx-web*, which means that this service is directly targeting *awx-web pods*, meaning that the UI/API is served through this. *NodePort = 31010* is properly configured and available for AWX UI access.

```
hasib@fedora-dev:~/devops-lab2-task3/awx/awx$ kubectl -n awx get svc awx-service -o wide
NAME          TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE   SELECTOR
awx-service   NodePort   10.43.173.121   <none>        80:31010/TCP   14h   app.kubernetes.io/component=awx,app.kubernetes.io/managed-by=awx-operator,app.kubernete
s.io/name=awx-web
```

Endpoint check - which pod does the service actually go to:

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get endpoints awx-service -o wide
```

The service was routed to *port 8052* of the *awx-web* pod IP (*10.42.x.x*) via ENDPOINTS *10.42.0.98:8052*. The deprecation warning was expected (*K8s v1.33* and Endpoints deprecated), but the output was valid. This means that the Service-to-Pod routing was correct.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get endpoints awx-service -o wide
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME          ENDPOINTS          AGE
awx-service   10.42.0.98:8052    12h
```

PoC:

Web Connectivity Test:

In this PoC web connectivity test, an HTTP request was sent directly to *NodePort* using *curl -I* to confirm that the AWX Web UI is indeed reachable.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ curl -I http://192.168.145.130:31010 | head
```

Getting *HTTP/1.1 200 OK* in Response means the service is live and serving the request properly, and *Server: nginx* shows that the AWX web front-end is being successfully served by nginx. This means that the AWX web service is up and running, and Kubernetes *NodePort* routing is working.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ curl -I http://192.168.145.130:31010 | head
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0  1044    0     0    0     0      0      0 --:--:--  0:00:01 --:--:--     0
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 31 Jan 2026 01:29:59 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 1044
Connection: keep-alive
Vary: Accept-Language, origin
Content-Language: en
X-API-Total-Time: 1.548s
X-API-Request-Id: ec124eae40534befa2e7db753e90619f
```

Admin Password Verification:

The AWX admin password was stored in a Kubernetes Secret. So, the password was decoded from the secret and verified. The *.data.password* field of the Kubernetes secret is base64 encoded. So, the real password can be seen by decoding it with *base64 -d*.

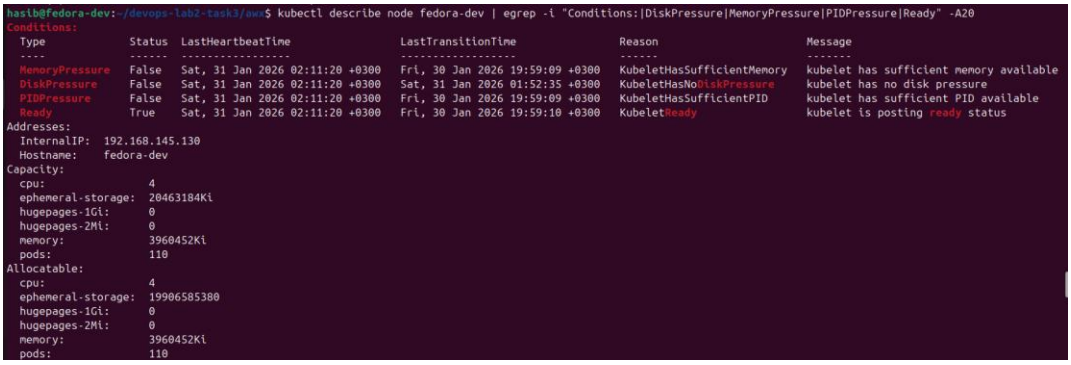
```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get secret awx-admin-password \
  -o jsonpath='{.data.password}' | base64 -d
<STRONG_ADMIN_PASSWORD>hasib@fedora-dev:~/devops-lab2-task3/awx$
```

Operational Issues & Resolution:

During deployment, several pods were evicted due to ephemeral storage pressure on a single-node cluster. Due to limited disk space and image garbage collection on the VM, some pods sometimes get stuck in the Evicted, *ContainerStatusUnknown*, Pending, or *Init:0/2 state*.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl describe node fedora-dev | egrep -i "Conditions:|DiskPressure|MemoryPressure|PIDPressure|Ready" -A20
```

The output of *kubectl* describe node fedora-dev shows that the node is currently *Ready=True* and *DiskPressure=False*, which means that the node is now in a healthy state. However, the event history shows *EvictionThresholdMet* and *FreeDiskSpaceFailed* warnings, which prove that disk pressure occurred earlier and that is when the pod eviction and container issue occurred. Later, after garbage collection and recovery, the node returned to normal.



```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl describe node fedora-dev | egrep -i "Conditions:|DiskPressure|MemoryPressure|PIDPressure|Ready" -A20
Conditions:
  Type             Status  LastHeartbeatTime                 LastTransitionTime                Reason                       Message
  ----             ------  -----------------                 ------------------                ------                       -------
  MemoryPressure   False   Sat, 31 Jan 2026 02:11:20 +0300   Fri, 30 Jan 2026 19:59:09 +0300   KubeletHasSufficientMemory   kubelet has sufficient memory available
  DiskPressure     False   Sat, 31 Jan 2026 02:11:20 +0300   Sat, 31 Jan 2026 01:52:35 +0300   KubeletHasNoDiskPressure     kubelet has no disk pressure
  PIDPressure      False   Sat, 31 Jan 2026 02:11:20 +0300   Fri, 30 Jan 2026 19:59:09 +0300   KubeletHasSufficientPID      kubelet has sufficient PID available
  Ready            True    Sat, 31 Jan 2026 02:11:20 +0300   Fri, 30 Jan 2026 19:59:10 +0300   KubeletReady                 kubelet is posting ready status
Addresses:
  InternalIP:  192.168.145.130
  Hostname:    fedora-dev
Capacity:
  cpu:                4
  ephemeral-storage:  20463184Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             3960452Ki
  pods:               110
Allocatable:
  cpu:                4
  ephemeral-storage:  19906585380
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             3960452Ki
  pods:               110
```



```
System Info:
  Machine ID:                 20b75429b1414e79afa2d0faf9c1d7ae
  System UUID:                dfae4d56-dc59-b173-4afd-c2a6c35d8ec8
--
  Normal   NodeHasDiskPressure    34m (x15 over 9h)     kubelet  Node fedora-dev status is now: NodeHasDiskPressure
  Warning  EvictionThresholdMet   32m (x71 over 9h)     kubelet  Attempting to reclaim ephemeral-storage
  Normal   NodeHasNoDiskPressure  24m (x19 over 21h)    kubelet  Node fedora-dev status is now: NodeHasNoDiskPressure
  Warning  FreeDiskSpaceFailed    19m                   kubelet  Failed to garbage collect required amount of images. Attempted to free 864166707 bytes, but only f
ound 468952208 bytes eligible to free.
```

Non-running containers identify (runtime-level check):

This command is run at the container runtime (containerd) level of *K3s* to identify which containers are running.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ sudo k3s crictl ps -a | awk '$4!="Running"{print $1}' | tail
```

crictl ps -a shows all containers (running + exited), and awk *'$4!="Running"{print $1}'* only shows the container IDs of non-running ones; tail shows the last few IDs quickly. The ability to extract non-running container IDs makes targeted cleanup easier and improves cluster stability.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ sudo k3s crictl ps -a | awk '$4!="Running"{print $1}' | tail
[sudo] password for hasib:
3568b48cbc7fe
45e7d3a2106f1
5d61d12676d0b
96ece85898561
c1594904026a7
bb8ecbd9e4e57
1f9f97201f591
6f8df3de49234
3e3fa52c257ee
8c297e7fc7ad5
```

Cleanup actions taken (space reclaim):

stopped containers cleanup:

The following command is used to clean up all stopped/exited containers at once, thereby reducing disk/ephemeral storage pressure by removing unnecessary containers accumulated in the runtime.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ sudo k3s crictl rm -a
```

Some containers being removed in the output means that the cleanup was successful. In some cases, it is expected that "*running, please stop it first*" will be displayed. This is because a running container cannot be removed directly with rm, it will not be deleted unless it is stopped first.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ sudo k3s crictl rm -a
96ece8589856133cc84b93e96bb9af84e6477cbb6746dcbb20f06b2e45ef7f05
3e3fa52c257ee10a9085d227f917e1c6f3ca8292143344784530dac65a343b13
container "1f9f97201f591544f7edcd548e1077066b3401bec6ad606d944cb234c7826208" is running, please stop it first
container "bb8ecbd9e4e57c9cd5378f54dd93f61d600a47dab20849b179138d58b3663c22" is running, please stop it first
container "5d61d12676d0b9efb2e48db2daa506b296dd22640e8468c7f3ef1cc782e644cc" is running, please stop it first
container "b3d4c15c0235f2150aadf92d2c27f267a0b13adfac00bdc007bd6fd8dee1fa72" is running, please stop it first
container "6f8df3de4923434d749214d2fa5f79419a8d28d9928eadc6ea789ea4c61cf4cb" is running, please stop it first
container "45e7d3a2106f146658e6c669163ab5fbf629613df79b158661a77a0dbe00d8d9" is running, please stop it first
container "c1594904026a71c81c0ce90bdb5d29de58d710623d467c20b442d39a7eaa0329" is running, please stop it first
container "8c297e7fc7ad5d14aeca703dd44906b138217595c10316cbd48419c97f928495" is running, please stop it first
container "e49d54e7d84f63c32a347affa523a2eb2ec98c11ec1dfbdcc3140ec2d9007353" is running, please stop it first
container "3568b48cbc7fee8d528de8d594fce1113b023cbd1e8ca541f2ec10fd6a2639aa" is running, please stop it first
```

The above command is run to clean up unused (dangling) container images. It deletes all images that are no longer being used by any running containers. This frees up disk / ephemeral storage and prevents image cache from becoming bloated.

Apt cache cleanup (disk space reduce):

This step performed system-level cleanup to reduce ephemeral disk space. The package cache was cleared by running apt-get clean and deleting */var/cache/apt/archives/** , and temporary files were removed by cleaning */tmp/** . The purpose of this step was to reduce unnecessary disk usage, so that eviction pressure from the *kubelet* would be reduced and the single-node lab cluster would be more stable.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ sudo k3s crictl rmi --prune
```

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ sudo apt-get clean -y
hasib@fedora-dev:~/devops-lab2-task3/awx$ sudo rm -rf /var/cache/apt/archives/*
hasib@fedora-dev:~/devops-lab2-task3/awx$ sudo rm -rf /tmp/*
```

Recovery: Rollout restart (pods fresh start):

After cleanup, *kubectl -n awx rollout restart* is used to replace the stuck/old state pods and create fresh pods. Restarting *awx-operator-controller-manager* allows the operator to reconcile again and restore the desired state properly, and restarting *awx-web* and *awx-task* clears the stuck *init/pending* state of the web/task component and allows it to start up again stably. The output showing all three "*restarted*" means that the restart was successful, and after that, the AWX web service is properly accessible again as a combined effect of cleanup + restart.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx rollout restart deploy awx-operator-controller-manager
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx rollout restart deploy awx-operator-controller-manager
deployment.apps/awx-operator-controller-manager restarted
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx rollout restart deploy awx-web
```

***deployment.apps/awx-web* restarted:**

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx rollout restart deploy awx-web
deployment.apps/awx-web restarted
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx rollout restart deploy awx-task
```

***deployment.apps/awx-task* restarted:**

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx rollout restart deploy awx-task
deployment.apps/awx-task restarted
```

After cleanup, AWX web service became accessible again.

Final Validation:

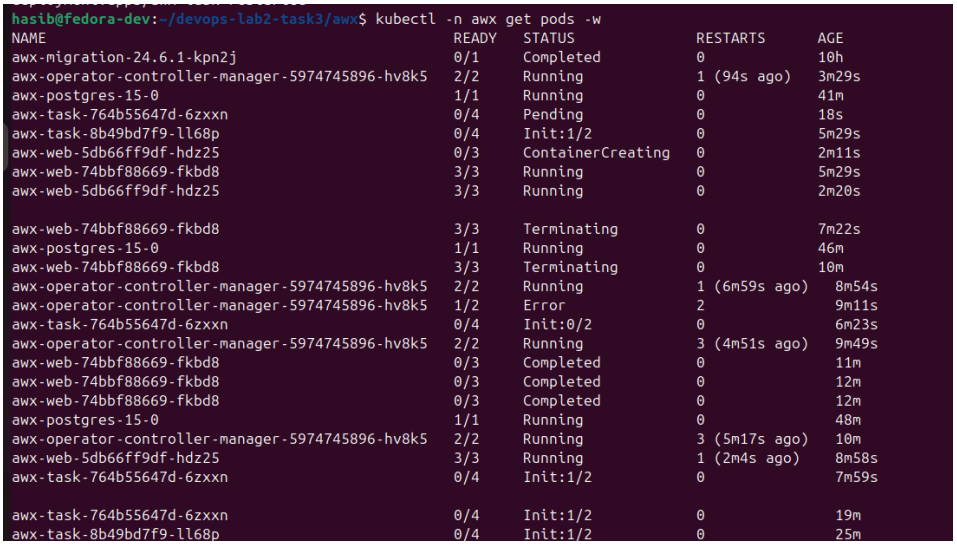
Whether the deployed AWX is running properly, whether the service is properly routed to the *awx-web pod*, and whether the NodePort/ClusterIP/Pod IP is web reachable at all levels are validated as final proof.

Pods live status monitor (real-time):

By running the following command with *-w* (watch), you can see the status changes of the pods live, such as Pending to Running, Creating to Running, or Terminating to New pod. When AWX Operator reconciles, it is normal for the old pod to be terminated and a new pod to be created, and this can be clearly seen in watch mode.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get pods -w
```

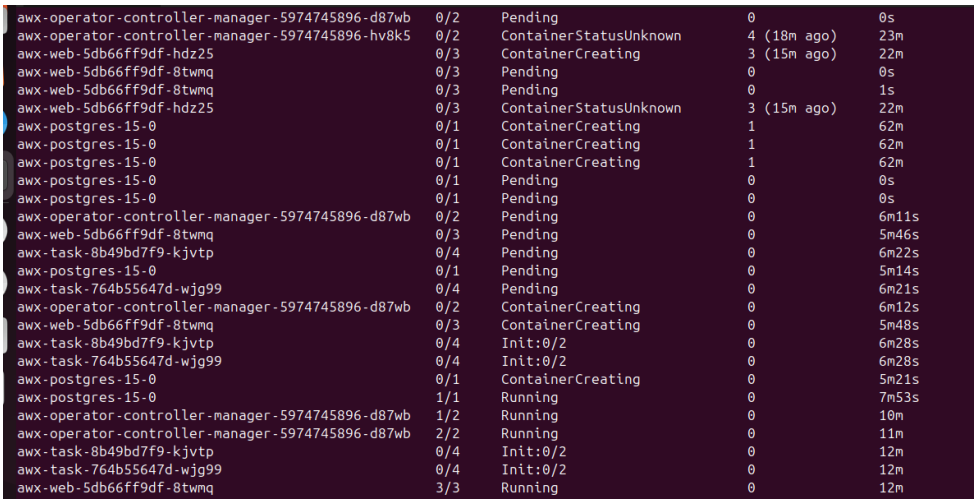
According to the output, *awx-migration-...* Completed means the DB migration has completed successfully, awx-postgres-15-0 Running means the database is up and running, and at least one pod of *awx-web* is 3/3 Running, indicating that the UI container set is up properly. *awx-operator-controller-manager* is Running, indicating that the operator is active, and *Pending/Init:1/2/Unknown* in *awx-task* is consistent with the ephemeral storage/disk pressure issue of the single-node cluster.



```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get pods -w
NAME                                               READY   STATUS              RESTARTS      AGE
awx-migration-24.6.1-kpn2j                         0/1     Completed           0             10h
awx-operator-controller-manager-5974745896-hv8k5   2/2     Running             1 (94s ago)   3m29s
awx-postgres-15-0                                  1/1     Running             0             41m
awx-task-764b55647d-6zxxn                          0/4     Pending             0             18s
awx-task-8b49bd7f9-ll68p                           0/4     Init:1/2            0             5m29s
awx-web-5db66ff9df-hdz25                           0/3     ContainerCreating   0             2m11s
awx-web-74bbf88669-fkbd8                           3/3     Running             0             5m29s
awx-web-5db66ff9df-hdz25                           3/3     Running             0             2m20s

awx-web-74bbf88669-fkbd8                           3/3     Terminating         0             7m22s
awx-postgres-15-0                                  1/1     Running             0             46m
awx-web-74bbf88669-fkbd8                           3/3     Terminating         0             10m
awx-operator-controller-manager-5974745896-hv8k5   2/2     Running             1 (6m59s ago)   8m54s
awx-operator-controller-manager-5974745896-hv8k5   1/2     Error               2               9m11s
awx-task-764b55647d-6zxxn                          0/4     Init:0/2            0               6m23s
awx-operator-controller-manager-5974745896-hv8k5   2/2     Running             3 (4m51s ago)   9m49s
awx-web-74bbf88669-fkbd8                           0/3     Completed           0               11m
awx-web-74bbf88669-fkbd8                           0/3     Completed           0               12m
awx-web-74bbf88669-fkbd8                           0/3     Completed           0               12m
awx-postgres-15-0                                  1/1     Running             0               48m
awx-operator-controller-manager-5974745896-hv8k5   2/2     Running             3 (5m17s ago)   10m
awx-web-5db66ff9df-hdz25                           3/3     Running             1 (2m4s ago)    8m58s
awx-task-764b55647d-6zxxn                          0/4     Init:1/2            0               7m59s

awx-task-764b55647d-6zxxn                          0/4     Init:1/2            0               19m
awx-task-8b49bd7f9-ll68p                           0/4     Init:1/2            0               25m
```



```
awx-operator-controller-manager-5974745896-d87wb   0/2     Pending                 0               0s
awx-operator-controller-manager-5974745896-hv8k5   0/2     ContainerStatusUnknown  4 (18m ago)     23m
awx-web-5db66ff9df-hdz25                           0/3     ContainerCreating       3 (15m ago)     22m
awx-web-5db66ff9df-8twmq                           0/3     Pending                 0               0s
awx-web-5db66ff9df-8twmq                           0/3     Pending                 0               1s
awx-web-5db66ff9df-hdz25                           0/3     ContainerStatusUnknown  3 (15m ago)     22m
awx-postgres-15-0                                  0/1     ContainerCreating       1               62m
awx-postgres-15-0                                  0/1     ContainerCreating       1               62m
awx-postgres-15-0                                  0/1     ContainerCreating       1               62m
awx-postgres-15-0                                  0/1     Pending                 0               0s
awx-postgres-15-0                                  0/1     Pending                 0               0s
awx-operator-controller-manager-5974745896-d87wb   0/2     Pending                 0               6m11s
awx-web-5db66ff9df-8twmq                           0/3     Pending                 0               5m46s
awx-task-8b49bd7f9-kjvtp                           0/4     Pending                 0               6m22s
awx-postgres-15-0                                  0/1     Pending                 0               5m14s
awx-task-764b55647d-wjg99                          0/4     Pending                 0               6m21s
awx-operator-controller-manager-5974745896-d87wb   0/2     ContainerCreating       0               6m12s
awx-web-5db66ff9df-8twmq                           0/3     ContainerCreating       0               5m48s
awx-task-8b49bd7f9-kjvtp                           0/4     Init:0/2                0               6m28s
awx-task-764b55647d-wjg99                          0/4     Init:0/2                0               6m28s
awx-postgres-15-0                                  0/1     ContainerCreating       0               5m21s
awx-postgres-15-0                                  1/1     Running                 0               7m53s
awx-operator-controller-manager-5974745896-d87wb   1/2     Running                 0               10m
awx-operator-controller-manager-5974745896-d87wb   2/2     Running                 0               11m
awx-task-8b49bd7f9-kjvtp                           0/4     Init:0/2                0               12m
awx-task-764b55647d-wjg99                          0/4     Init:0/2                0               12m
awx-web-5db66ff9df-8twmq                           3/3     Running                 0               12m
```

Service exists + NodePort confirm:

This step verifies the *awx-service* and ensures that the AWX Web UI is correctly exposed with *NodePort*.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get svc awx-service -o wide
```

The output showing *TYPE: NodePort* and *PORT(S): 80:31010/TCP* means that the service is running on *port 80* inside the cluster, and can be accessed from the outside using the node IP using *port 31010*. The presence of *awx-web* in the SELECTOR means that the service is directly targeting the *awx-web pods*.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get svc awx-service -o wide
NAME          TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE   SELECTOR
awx-service   NodePort   10.43.173.121   <none>        80:31010/TCP   12h   app.kubernetes.io/component=awx,app.kubernetes.io/managed-by=awx-operator,app.kubernetes.io/name=awx-web
```

Endpoint binding confirm (service - pod mapping):

This step confirms pod routing from the service. The output of *kubectl* get endpoints *awx-service* shows *10.42.0.98:8052*, meaning that the *awx-service* now has a live pod endpoint.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get endpoints awx-service -o wide
```

The service is correctly routed to the IP and port of the *awx-web pod*.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get endpoints awx-service -o wide
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME          ENDPOINTS          AGE
awx-service   10.42.0.98:8052    12h
```

NodePort/TargetPort/Endpoint proof with Service describe:

This output proves the entire routing chain in one place.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx describe svc awx-service | sed -n '/Selector:/,/Endpoints:/p'
```

Here we see that the service type is *NodePort,* when the client hits *node:31010* from outside, it becomes service port 80 and maps to *targetPort 8052*, and finally the traffic reaches the *awx-web pod* *(10.42.0.98:8052*).

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx describe svc awx-service | sed -n '/Selector:/,/Endpoints:/p'
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
Selector:                 app.kubernetes.io/component=awx,app.kubernetes.io/managed-by=awx-operator,app.kubernetes.io/name=awx-web
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.43.173.121
IPs:                      10.43.173.121
Port:                     http  80/TCP
TargetPort:               8052/TCP
NodePort:                 http  31010/TCP
Endpoints:                10.42.0.98:8052
```

The output below shows *NodePort, TargetPort,* and *Endpoints* together.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get pods -l app.kubernetes.io/name=awx-web --show-labels
```

Here, if the client requests *node:31010* from outside, it maps to *port 80* of the service and *targetPort 8052*, and finally the traffic reaches pod endpoint *10.42.0.98:8052*. Label filter confirms that there are pods matching the *awx-web* label. There is a running pod, so the service endpoint is alive and the Unknown pod was stuck in the previous pressure/eviction/restart.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx get pods -l app.kubernetes.io/name=awx-web --show-labels
NAME                       READY   STATUS                RESTARTS   AGE   LABELS
awx-web-5db66ff9df-hmnts   3/3     Running               0          13m   app.kubernetes.io/component=awx,app.kubernetes.io/managed-by=awx-operator,app.kubernet
es.io/name=awx-web,app.kubernetes.io/operator-version=2.19.1,app.kubernetes.io/part-of=awx,app.kubernetes.io/version=24.6.1,pod-template-hash=5db66ff9df
awx-web-5db66ff9df-jb926   0/3     ContainerStatusUnknown   3          22m   app.kubernetes.io/component=awx,app.kubernetes.io/managed-by=awx-operator,app.kubernet
es.io/name=awx-web,app.kubernetes.io/operator-version=2.19.1,app.kubernetes.io/part-of=awx,app.kubernetes.io/version=24.6.1,pod-template-hash=5db66ff9df
```

Pod IP direct test:

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ curl -I http://10.42.0.98:8052 | head
```

In this direct Pod IP test, the IP and port of the *awx-web* pod were hit directly using curl. Getting *HTTP/1.1 200 OK* and Server: nginx means that the application itself responded correctly.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ curl -I http://10.42.0.98:8052 | head
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0  1044    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 31 Jan 2026 01:24:25 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 1044
Connection: keep-alive
Vary: Accept-Language, origin
Content-Language: en
X-API-Total-Time: 0.563s
X-API-Request-Id: 9764a5faa33a4a7596a38bde7f20c95f
```

Service ClusterIP test:

In this test, a request was sent directly using the IP and port of the *ClusterIP* service.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ curl -I http://10.43.173.121:80 | head
```

Getting *HTTP/1.1 200 OK* and *Server: nginx* means the service has correctly routed traffic to the *awx-web* pod.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ curl -I http://10.43.173.121:80 | head
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0  1044    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 31 Jan 2026 01:24:51 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 1044
Connection: keep-alive
Vary: Accept-Language, origin
Content-Language: en
X-API-Total-Time: 0.496s
X-API-Request-Id: f6c08517cd7e42099ecfe149932d832b
```

Port-forward test (debug/alt access):

This port-forward test is done to verify debug or alternative access. Here, *port 8052* of the *awx-web* pod is forwarded directly to *port 8043* of the local/node.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx port-forward pod/awx-web-5db66ff9df-hmnts 8043:8052 --address 0.0.0.0
```

If there is a problem with the *service/NodePort,* port-forwarding can quickly confirm that the application itself is fine. This means that the pod can be accessed directly without having to bypass the service or *NodePort*.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ kubectl -n awx port-forward pod/awx-web-5db66ff9df-hmnts 8043:8052 --address 0.0.0.0
Forwarding from 0.0.0.0:8043 -> 8052
Handling connection for 8043
```

This step is directly verified by running curl on the local port-forwarded port. Getting an *HTTP/1.1 200 OK* means that port-forward is working properly and the AWX UI is being reached locally.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ curl -I http://127.0.0.1:8043 | head
```

Even if the service or *NodePort* is completely bypassed, the application is reachable, meaning the *awx-web* pod is healthy and the UI is served properly.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ curl -I http://127.0.0.1:8043 | head
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0  1044    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 31 Jan 2026 01:26:59 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 1044
Connection: keep-alive
Vary: Accept-Language, origin
Content-Language: en
X-API-Total-Time: 0.276s
X-API-Request-Id: 84c94599d589487993a5af4c1a0f1334
```

***NodePort* final public path:**

This last test verifies the AWX Web UI directly using *Node IP + NodePort*. Getting *HTTP/1.1 200 OK* in the above command means that Kubernetes *NodePort* routing is working properly and the service is reachable from outside the node. The AWX Web UI is successfully externally accessible, meaning that the deployment, service configuration, and network path all work properly end-to-end.

```
hasib@fedora-dev:~/devops-lab2-task3/awx$ curl -I http://192.168.145.130:31010 | head
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0  1044    0     0    0     0      0      0 --:--:--  0:00:01 --:--:--     0
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 31 Jan 2026 01:29:59 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 1044
Connection: keep-alive
Vary: Accept-Language, origin
Content-Language: en
X-API-Total-Time: 1.548s
X-API-Request-Id: ec124eae40534befa2e7db753e90619f
```

Conclusion for Bonus 2:

Bonus Task-2 successfully deploys Ansible AWX on a *K3s* (single-node) Kubernetes cluster using the official GitHub repository and AWX Operator and shows a working Proof of Concept (*PoC*). First, install K3s and confirm node readiness with *kubectl* get nodes, then set up the necessary tools (*Git, kubectl*) and clone the AWX repo. To simplify isolation and management in Kubernetes, the awx namespace is used and the correct placement of the Operator is verified by looking at AWX CRDs (*awxs.awx.ansible.com* etc.). Then, the admin password is managed securely through the Kubernetes Secret (*awx-admin-password*) instead of keeping it in plaintext, and when the AWX Custom Resource (*awx-deploy.yml*) is applied, the Operator automatically creates the necessary components including *awx-web, awx-task, awx-postgres, awx-migration*, where *awx-web* Running, *awx-postgres* Running and *awx-migration* Completed are strong evidence of deployment success. The AWX service is exposed as *NodePort (80:31010/TCP*). The endpoints and service describe output clearly show that the client request first reaches the node's *port 31010*, then is mapped to *targetPort 8052* by the service and is correctly routed to the *awx-web* pod. As a PoC, *curl -I http://192.168.145.130:31010* returns *HTTP/1.1 200 OK* (with *Server: nginx*), which shows that the AWX web interface is actually reachable. During deployment, some pods were Pending/Evicted/Unknown due to ephemeral storage pressure due to the single-node cluster. This was detected by the node describe event history and recovered by *crictl* cleanup, image prune, apt cache cleanup, then rollout restart, after the cleanup, AWX web is stably accessible again. Finally, end-to-end routing and application health were verified by receiving 200 OK responses at all levels of Pod IP, *ClusterIP*, port-forward, and *NodePort*.

References

[1] Ansible Documentation, "*Ansible User Guide*," Red Hat, 2025. Available: https://docs.ansible.com/ansible/latest/user_guide/index.html

[2] Ansible Documentation, "*Ansible Directory Layout*," Red Hat, 2025. Available: https://docs.ansible.com/ansible/latest/reference_appendices/config.html

[3] Ansible Documentation, "*Ansible Configuration Settings (ansible.cfg)*," Red Hat, 2025. Available: https://docs.ansible.com/ansible/latest/reference_appendices/config.html

[4] Ansible Documentation, "*Using Variables in Ansible*," Red Hat, 2025. Available: https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_variables.html

[5] Ansible Documentation, "*Variable Precedence*," Red Hat, 2025. Available: https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_variables.html#understanding-variable-precedence

[6] Ansible Documentation, "*Roles - Best Practices*," Red Hat, 2025. Available: https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_reuse_roles.html

[7] Red Hat, "*Ansible Best Practices Guide*," Red Hat, 2025. Available: https://www.redhat.com/en/topics/automation/ansible-best-practices

[8] Kubernetes Documentation, "*Kubernetes Official Documentation*," The Linux Foundation, 2025. Available: https://kubernetes.io/docs/home/

[9] Rancher Labs, "*K3s - Lightweight Kubernetes*," Rancher, 2025. Available: https://docs.k3s.io/

[10] Ansible AWX Documentation, "*AWX - Open Source Ansible Automation Platform*," Ansible Community, 2025. Available: https://ansible.readthedocs.io/projects/awx/en/latest/

[11] AWX Operator Documentation, "*AWX Operator Installation Guide*," Ansible Community, 2025. Available: https://ansible.readthedocs.io/projects/awx-operator/en/latest/

[12] GitHub, "*AWX Operator Official Repository*," Ansible Community, 2025. Available: https://github.com/ansible/awx-operator

[13] GitHub, "*AWX Official Source Repository*," Ansible Community, 2025. Available: https://github.com/ansible/awx

[14] Kubernetes Documentation, "*Services - NodePort*," The Linux Foundation, 2025. Available: https://kubernetes.io/docs/concepts/services-networking/service/#nodeport

[15] Kubernetes Documentation, "*Pods, Deployments and ReplicaSets*," The Linux Foundation, 2025. Available: https://kubernetes.io/docs/concepts/workloads/

[16] Ubuntu Documentation, "*Ubuntu Server Guide*," Canonical Ltd., 2025. Available: https://ubuntu.com/server/docs

[17] Fedora Documentation, "*Fedora Server Documentation*," Fedora Project, 2025. Available: https://docs.fedoraproject.org/en-US/fedora-server/

[18] Red Hat, "*Infrastructure Automation with Ansible*," Red Hat, 2025. Available: https://www.redhat.com/en/technologies/management/ansible